Fundamentos de Ethereum

PID_00284213

Josep Lluís de la Rosa Esteva
Alberto Ballesteros Rodríguez

Tiempo mínimo de dedicación recomendado: 4 horas



Universitat Oberta de Catalunya

Josep Lluís de la Rosa Esteva **Alberto Ballesteros Rodríguez**

El encargo y la creación de este recurso de aprendizaje UOC han sido coordinados por el profesor: Victor Garcia Font

Primera edición: septiembre 2021
© de esta edición, Fundació Universitat Oberta de Catalunya (FUOC)
Av. Tibidabo, 39-43, 08035 Barcelona
Autoría: Josep Lluís de la Rosa Esteva, Alberto Ballesteros Rodríguez
Producción: FUOC
Todos los derechos reservados

Ninguna parte de esta publicación, incluido el diseño general y la cubierta, puede ser copiada, reproducida, almacenada o transmitida de ninguna forma, ni por ningún medio, sea este eléctrico, mecánico, óptico, grabación, fotocopia, o cualquier otro, sin la previa autorización escrita del titular de los derechos.

Índice

Introducción..... 5

Objetivos..... 6

1. Hacia una web descentralizada, o Web 3.0..... 7

2. ¿Es Ethereum diferente de Bitcoin?..... 10
2.1. Comparando Ethereum con Bitcoin 12
2.2. Una máquina Turing completa 12

3. Componentes de la arquitectura de Ethereum..... 14
3.1. Clientes 15
3.1.1. Cuentas de Ethereum 16
3.2. Máquina virtual de Ethereum 17
3.2.1. El *bytecode* de la EVM 19
3.3. Protocolo de consenso en Ethereum 22
3.3.1. Ethereum 1.0: Ethash 25
3.3.2. Ethereum 2.0 26
3.4. Redes de Ethereum 28
3.4.1. Redes públicas 28
3.4.2. Redes privadas 28
3.5. Ecosistema de Ethereum 29
3.5.1. Swarm 29
3.5.2. Ethereum Name Service 30
3.5.3. Oráculos 32
3.5.4. Soluciones de segunda capa 32

4. Las transacciones en Ethereum..... 35
4.1. Estructura de una transacción 35
4.2. Los pagos en Ethereum: ether y gas 35
4.2.1. Sobre los límites del gas 36
4.2.2. Mecanismo de valoración del precio de gas 40
4.2.3. Críticas al gas de Ethereum 41
4.3. Consulta de la ejecución de una transacción 42

5. Creación, publicación y ejecución de contrato inteligente..... 43

6. Las DApps (aplicaciones descentralizadas)..... 45
6.1. Ejemplos de DApps 47
6.1.1. Intercambios descentralizados (DEX) 48
6.1.2. Organizaciones autónomas descentralizadas (DAO) 48

Resumen..... 50

Glosario..... 51

Bibliografía..... 52

Introducción

En 2009 aparece bitcóin, una criptomoneda que habilita un sistema de pagos digitales descentralizado gracias a su tecnología subyacente, la *blockchain*. A partir de aquí, investigadores de todo el mundo analizan las posibilidades de la *blockchain* para crear muchos otros servicios y modelos de negocio descentralizados y sin intermediarios más allá de los pagos digitales. En un primer momento, cada nuevo servicio que se diseña propone desplegar una nueva plataforma de *blockchain* adaptándola a las características de su aplicación. Ethereum supone un gran cambio en este contexto, ya que se diseña para ser una computadora universal donde se pueda ejecutar cualquier aplicación descentralizada. Es decir, en vez de necesitar una *blockchain* diferente para cada servicio, Ethereum pretende ser una plataforma donde los desarrolladores solo tengan que preocuparse de desplegar sus aplicaciones y compartan una *blockchain* en la que se almacenen todas las transacciones independientemente de la aplicación para la que se destinan.

En este módulo revisaremos el concepto de Web 3.0, que pretende rediseñar la web convencional de hoy en día e ir hacia una arquitectura más descentralizada donde los usuarios puedan recuperar el control de sus datos personales, con menos intermediarios y menos puntos únicos de fallo.

Una pieza clave para habilitar esta Web 3.0 es la tecnología *blockchain* y, en particular, Ethereum, al ser la plataforma más utilizada en estos momentos. En este módulo describiremos los componentes básicos de Ethereum y veremos cómo funciona este sistema para desplegar los contratos inteligentes, que son los programas informáticos que corren encima de la *blockchain* y que dan las funcionalidades concretas a las diferentes aplicaciones descentralizadas (DApps).

Objetivos

En este módulo didáctico se fijan los siguientes objetivos:

1. Entender el concepto y los componentes básicos de la Web 3.0.

2. Entender la arquitectura y el protocolo de Ethereum.

3. Adquirir los conocimientos teóricos básicos para así poder interactuar con Ethereum para desarrollar y desplegar contratos inteligentes.

1. Hacia una web descentralizada, o Web 3.0

A raíz de que los usuarios durante años hemos confiado nuestra información a entidades en Internet, nos hemos dado cuenta del peligro que supone esta acción, por ejemplo, por posibles hackeos o por el mal uso que los proveedores puedan hacer de nuestros datos personales. Por esto, podemos afirmar que el actual modelo de gobierno de la información por parte de estas entidades es un modelo que entraña ciertos riesgos. Esto se debe a que muchas organizaciones tienden a un modelo de negocio centralizado que requiere conocer la máxima información posible acerca de sus usuarios.

Para poder movernos hacia un Internet más descentralizado, las actuales tecnologías y protocolos utilizados que han servido para ofrecernos el Internet que conocemos hoy en día, como pueden ser SMTP, HTTP, HTML, FTP, etc., sobre los que se sustentan la mayoría de los servicios que utilizamos actualmente, deben ser rediseñados acorde a una nueva comprensión de la interacción entre la tecnología y la sociedad.

Aquí aparece la denominada Web 3.0 (Wood, 2014a, 2014b), que se trata de un rediseño de la web que utilizamos hoy en día. Con la Web 3.0, se hace la siguiente subdivisión en cuanto a la información:

- La información que se considera pública se publica.
- La información que se considera privada no se publica y se mantiene en secreto.
- La información sobre la que se está de acuerdo se publica en una estructura de datos compartida y consensuada públicamente, generalmente en forma de cadena de bloques, o *blockchain*. Como veremos más adelante, de forma general, por su similitud con los libros de contabilidad, donde solo pueden añadirse datos nuevos y nunca eliminar o modificar entradas ya existentes, a esta estructura de datos la llamamos *ledger* ('libro mayor', en español).

Además, la comunicación siempre sucederá a partir de canales cifrados y mediante identidades seudónimas de los *endpoints*, no con identidades o elementos rastreables como puede ser una IP.

La Web 3.0 consta de los siguientes componentes:

- **Publicación de contenido estático**: un sistema que básicamente se basa en la solicitud de cierta información por parte del usuario a partir de una dirección y la devolución de dicha información por parte del sistema. Información estática, es decir, siempre la misma al acceder a la misma dirección. Una vez descargada la información, se puede estar seguro de que es correcta, dado que la dirección utilizada es intrínseca a esta información.

Por ejemplo, una dirección a un recurso puede crearse a partir del valor *hash* de ese recurso. Además, este sistema de publicación de contenido estático debe permitir el envío de nueva información en cualquier momento. Una implementación existente de esta tecnología es BitTorrent, y en la Web 3.0 se utiliza para alojar y descargar información estática y normalmente de gran tamaño, siempre y cuando se quiera compartir dicha información. Para mantener esta información alojada, se deben añadir sistemas de incentivos para los usuarios que decidan mantener y compartir información.

- **Mensajes dinámicos**: sistema de mensajería a bajo nivel basado en identidades seudónimas que se utiliza para comunicarse entre personas de la red. Se utiliza criptografía de clave pública-privada para garantizar la privacidad de los mensajes y que su recepción sea segura. Los mensajes tendrán un tiempo de vida, lo que hace que se pueda decidir el tiempo que se desea que los mensajes se mantengan visibles para las distintas partes. La comunicación entre pares (*peers,* en inglés) en este sistema se basa en un sistema de reputación que define cada par respecto de sus conexiones, quedándose el par con aquellos pares que brinden la información más valiosa. En la Web 3.0, esta parte permite a los pares comunicarse, actualizarse y autoorganizarse en tiempo real, de manera que pueden publicar información cuya procedencia no necesita ser intrínsecamente confiable o referida posteriormente.

- **Transacciones sin confianza** (*trustless,* en inglés): esta parte de la Web 3.0 es el motor de consenso, es decir, el medio para acordar las reglas de interacción que se ejecutarán de manera automática e irrevocable en el futuro sobre las posibles interacciones. El hecho de que el resultado de un acuerdo entre partes se pueda comprobar por todos los demás integrantes de la red es un factor clave para que se cree un sistema fuerte que no admita cambios sobre contratos o se ignoren las reglas establecidas en dichos contratos. Gracias a este motor de consenso, se registrará cualquier publicación y alteración de la información mediante un sistema de procesamiento de transacciones, como puede ser Ethereum. Actualmente, la web que conocemos no aborda este aspecto de consenso, sino que la confianza se basa en autoridades centrales, como entidades u organizaciones.

- **Interfaz de usuario**: este elemento de la Web 3.0 es el que tiene un aspecto más similar al de la web que conocemos hoy en día, ya que se trata de la interfaz de usuario que se nos presenta a partir del navegador. Sea una aplicación descentralizada, una página o sitio web, el navegador entregará una interfaz tal y como la conocemos hoy en día a partir de una URL introducida previamente en la barra de dirección. Aunque en la Web 3.0, el sistema de resolución de nombres no se basa en direcciones como las que conocemos hoy en día, sino más bien en el uso de valores *hash*, como se ha comentado anteriormente, para poder tener direcciones unívocas representando a los recursos publicados. Utilizando un sistema de publicación

de información como el mencionado anteriormente (por ejemplo, BitTorrent), podemos alojar colecciones de archivos utilizados para ofrecer la interfaz al usuario, parte estática de la web, como imágenes, estilos, *scripts*, etc. Asimismo, el contenido dinámico se atiende por medio de otros canales de comunicación, como por ejemplo por medio de las API ofrecidas con el motor de consenso, siempre y cuando el contenido que se consulte u ofrezca sea público. Para contenido privado, se deberán utilizar canales de comunicación *peer-to-peer* (P2P).

La Web 3.0 busca eliminar la necesidad de utilizar servidores centralizados y se aleja del modelo tradicional de URL cliente-servidor y el uso de direcciones https://. Para ello, Ethereum presenta un conjunto de tecnologías subyacentes que pretenden abarcar no solo elementos relacionados con la lógica de las aplicaciones, como son la ejecución de contratos inteligentes en su *blockchain*, sino también ofrecer almacenamiento distribuido a partir del protocolo Swarm y comunicaciones dinámicas.

Ethereum es una de las plataformas más prominentes para movernos hacia una Web 3.0. Actualmente, en esta plataforma las interacciones que realizan los usuarios se llevan a cabo de forma seudónima, segura y en un entorno *trustless*, que no requiere de un tercero de confianza para asegurar el correcto funcionamiento del sistema. Sin embargo, no todas las aplicaciones que se ejecutan en Ethereum pueden utilizar tan solo elementos de este ecosistema. En muchas ocasiones se requieren elementos externos, como por ejemplo datos provenientes de fuentes de datos convencionales. En estos casos, cuando se requiere la intervención de terceros, los desarrolladores de aplicaciones intentan minimizar las operativas que recaigan en una sola entidad para evitar posibles problemas de centralizar parte de un proceso (por ejemplo, puntos únicos de fallo, posible censura o corrupción por parte de un operador). Aunque una aplicación no sea 100 % descentralizada, actuando de esta forma se crean aplicaciones donde la confianza que deba depositarse en terceras partes sea reducida.

En los próximos apartados presentaremos los componentes esenciales de Ethereum, que, como hemos dicho, puede ser una pieza clave para ir hacia una Web 3.0. Posteriormente, en este módulo veremos otras herramientas de este ecosistema y lo culminaremos introduciendo las DApps, las aplicaciones descentralizadas.

2. ¿Es Ethereum diferente de Bitcoin?

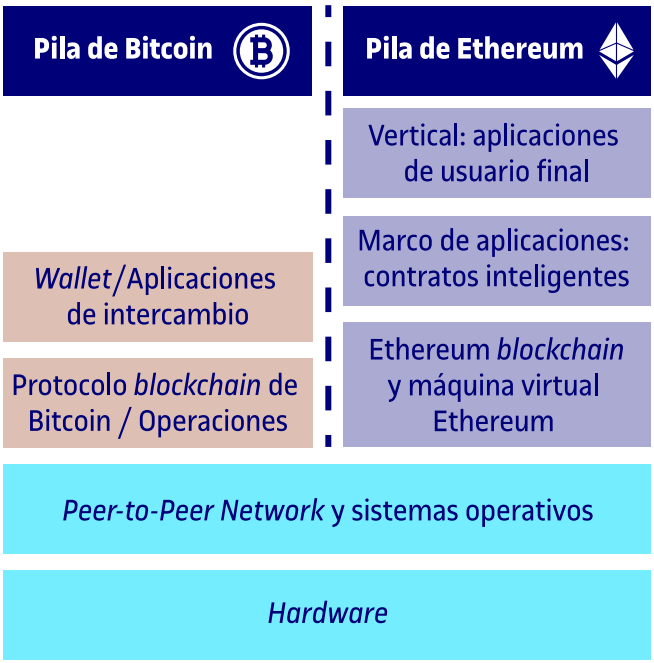
La idea de *blockchain* que tenemos ha cambiado poco después de una década de su concepción por Satoshi Nakamoto (2008), y los **contratos inteligentes** (*smart contracts,* en inglés) son uno de los cambios más notables. Los contratos inteligentes pueden hacer digitalmente **cumplir la lógica de un acuerdo** (quizás también un acuerdo contractual) **sin la participación de terceros** o, según lo declarado por Nick Szabo (1996), son «un protocolo de transacción computarizado que ejecuta los términos de un contrato».

Una transacción puede tener lugar en el *ledger* con las reglas establecidas en un contrato inteligente, el resultado siempre tiene que ser el mismo, independientemente de quién ejecute la lógica cuando se le proporciona la misma entrada (el mismo *output* para un mismo *input*), lo que produce resultados deterministas que cuentan con el consenso de los participantes de dicha *blockchain*.

Recordemos que un *ledger* ('libro contable', en español) es la estructura de información que se distribuye en la tecnología de libro distribuido (*distributed ledger technology*, o DLT), y que en una *blockchain* se implementa con una cadena de bloques. La denominación de *ledger* se debe a la similitud con los libros contables, donde se añaden nuevos apuntes contables para modificar los balances y nunca se eliminan o modifican datos ya introducidos.

Los contratos inteligentes pueden verse como una puerta de entrada de interacción con la *blockchain*, mediante llamadas a funciones (el código) y datos en el contrato inteligente y, como se ha dicho, allanan el camino para que no solo se registren transacciones en el *ledger*, sino que utilizando contratos inteligentes también es posible implementar la lógica para casos específicos que implican almacenamiento de información, realizar cálculos, compartir estados, editar estados y proporcionar condiciones de acceso. Esto se refleja en la figura 1, donde vemos como en Ethereum se añade una máquina virtual y una capa de programación de contratos inteligentes respecto a la *blockchain* original.

Figura 1. Comparativa entre las pilas de Bitcoin y de Ethereum



Fuente:https://www.coursera.org/lecture/blockchain-basics/ethereum-blockchain-smart-contracts-M2Yhb

La primera referencia de entorno de ejecución de contratos inteligentes es el sistema **Ethereum**, que lo definió Vitalik Buterin en 2013 en el *white paper* «A next-generation *smart contract* and decentralized application platform» (Buterin, 2013). Este *white paper* junto con el *white paper* de Satoshi Nakamoto de 2008, donde se hacía una primera especificación de Bitcoin, han supuesto una disrupción del estado del arte en la descentralización de la computación. A partir de entonces, la publicación de *white papers* por miles de iniciativas en DLT ha sido la manera normal de impulsar los nuevos proyectos en este entorno.

Ethereum usa una máquina de estado determinista, pero prácticamente ilimitada, que consta de un estado accesible globalmente y una máquina virtual que aplica cambios a ese estado. Según Andreas Antonopoulos (2018a), Ethereum se describe a menudo como «la computadora mundial», que se refiere a que con la plataforma Ethereum se pueden crear aplicaciones **descentralizadas** y globales con funciones económicas integradas.

En resumen, desde un punto de vista de uso práctico, Ethereum es una infraestructura informática global y descentralizada de código abierto que ejecuta programas llamados contratos inteligentes, o *smart contracts*. Ethereum, utiliza una cadena de bloques para sincronizar y almacenar los cambios de estado del sistema. Como se verá más adelante, Ethereum usa una criptomoneda llamada **ether (ETH)** para medir y limitar los costos de los recursos de ejecución.

2.1. Comparando Ethereum con Bitcoin

Ethereum comparte muchos elementos comunes con otros sistemas basados en cadenas de bloques **abiertas** como Bitcoin: una red P2P que conecta a los participantes, un algoritmo de consenso bizantino tolerante a fallos para la sincronización de actualizaciones de estado (de la cadena de bloques), el uso de primitivas criptográficas, como firmas digitales y *hashes*, y una moneda digitales (*ether*, en el caso de Ethereum; y bitcóin, en el caso de Bitcoin).

Aunque es posible realizar pagos utilizando *ethers*, es necesario destacar que el propósito principal de Ethereum no es ser una red de pago de moneda digital. La moneda digital *ether* es necesaria para la ejecución de contratos inteligentes en Ethereum, es decir, que además de ser medio de pago, el *ether* es necesario para pagar por el uso de la plataforma Ethereum como computadora mundial.

Dicho eso, Bitcoin y Ethereum proporcionan interfaces de programación para contratos inteligentes donde muchas funcionalidades, como micropagos, cuentas conjuntas, etc., se pueden implementar en ambas plataformas. Sin embargo, la filosofía entre los dos es diferente. Ethereum favorece la generalidad y la expresividad, mientras que Bitcoin está diseñado para ser más seguro y tener una superficie de ataques más pequeña, con el inconveniente de que ofrece menos funcionalidades programáticas que Ethereum.

Los contratos inteligentes de Ethereum se implementan con un lenguaje «Turing completo», como por ejemplo Solidity; a diferencia de Bitcoin, que tiene un lenguaje de *scripting*, secuencias de instrucciones, que es un lenguaje limitado en comparación. Ethereum, está diseñado para ser una cadena de bloques programable de propósito general que ejecuta una máquina virtual capaz de ejecutar código de complejidad arbitraria e ilimitada. Mientras que el lenguaje de secuencia de instrucciones de Bitcoin está, intencionalmente, limitado a una simple evaluación de verdadero o falso de las condiciones de gasto y evita toda posibilidad de usar bucles; el lenguaje de Ethereum es Turing completo, lo que significa que Ethereum puede funcionar como una computadora de propósito general. A continuación, damos más detalles sobre este tipo de lenguajes.

2.2. Una máquina Turing completa

El matemático Alan Turing imaginó una máquina en 1936 que podría resolver cualquier problema (matemático). Una computadora es Turing completa si puede resolver cualquier problema que una máquina de Turing pueda, dado

un algoritmo apropiado, el tiempo y la memoria necesarios. En la práctica, lo más relevante sobre un lenguaje catalogado como Turing completo es que se pueden programar bucles.

Bitcoin tiene un lenguaje de secuencias de instrucciones, *scripting*, que esencialmente permite ejecutar transacciones, donde cada transacción está compuesta por las instrucciones que describen exactamente cómo se moverán los bitcoines de una dirección a otra. En la *blockchain* de Bitcoin lo que se guardan son básicamente las transacciones y unos pocos metadatos limitados a pocos *bytes*. En cambio, Ethereum, además de almacenar las transacciones, adicionalmente puede almacenar estados arbitrarios asociados con cada contrato inteligente, y sus transiciones se verifican en la cadena de bloques según lo definido por las reglas establecidas por el contrato.

Los lenguajes de programación de Ethereum se ejecutan en varios nodos, y los resultados de sus ejecuciones consolidados a partir de la máquina virtual de Ethereum (EVM, *ethereum virtual machine*) se escriben en la cadena de bloques. Dichos lenguajes se asemejan a procesos basados en agentes inteligentes donde cada proceso de contrato inteligente se define como un comportamiento que reacciona a cada entrada (en Ethereum, mediante transacciones) y modifica un estado local (del contrato inteligente) y envía mensajes a otros agentes (contrato inteligente), y así sucesivamente, resolviendo en su conjunto una solución.

Por lo tanto, una gran ventaja de los lenguajes usados en Ethereum es su propósito general. En cambio, un gran inconveniente es que son propensos a errores (Miller y Bentov, 2016). Mientras tanto, el lenguaje de *scripting* de Bitcoin se centra principalmente en la verificación de firmas, no soporta bucles y, gracias a su simplicidad, es mucho menos propenso a errores.

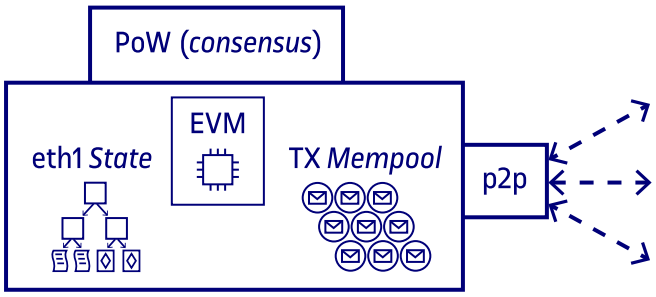
Para finalizar, hay que remarcar que aunque los lenguajes de Ethereum son Turing completos, para evitar ejecuciones interminables, **los tiempos de ejecución están limitados por tarifas de transacción**. Para la ejecución de las instrucciones se requiere que los usuarios paguen el llamado **gas**, lo que impide que un contrato inteligente se quede encallado infinitamente en un bucle. Aunque este es un concepto que se tratará a lo largo del módulo, en Rosic (2020) se puede encontrar una descripción detallada acerca del gas.

3. Componentes de la arquitectura de Ethereum

Ethereum es una plataforma *blockchain open source* y descentralizada. Dicha plataforma se compone de miles de nodos desplegados alrededor del mundo que forman una red con una topología P2P.

A diferencia de lo que pasa con las arquitecturas cliente-servidor convencionales, en una red P2P los participantes se comportan como iguales entre sí y actúan a la vez como cliente y servidor respecto a los demás participantes. Dependiendo del tipo de participación que se desee ejercer, un usuario podrá elegir entre varios tipos de nodos que desplegar. La figura 2 muestra una representación simplificada de un nodo, y en el apartado 3.1 se describe en más profundidad los diferentes tipos de nodos.

Figura 2. Representación simplificada de un nodo de Ethereum



Fuente: https://ethereum.org/en/developers/docs/nodes-and-clients/

Hay que tener presente que son los nodos de la red los que de forma conjunta ofrecen las diferentes funcionalidades de Ethereum. Como ya hemos comentado anteriormente, una de las principales funcionalidades de Ethereum es la ejecución de contratos inteligentes. Para ello, los contratos inteligentes se despliegan en la red de Ethereum mediante los nodos, los cuales implementan la máquina virtual Ethereum (EVM), que se encarga de ejecutar las funciones de estos contratos inteligentes. El apartado 3.2 da más detalles sobre cómo se lleva a cabo dicha ejecución. La ejecución de las funciones de los contratos y, en general, la modificación del estado del sistema se realiza mediante transacciones que reciben los nodos y que guardan en una memoria temporal llamada Mempool, hasta que las transacciones se ejecutan y su resultado se guarda de forma consensuada entre todos los nodos de la red en la *blockchain* mediante el protocolo PoW (*Proof-of-Work*). De esta forma, la *blockchain* mantiene el registro del estado global del sistema.

En el apartado 3.3 se dan más detalles sobre la *blockchain* y el protocolo de consenso. Hay que tener en cuenta que en este apartado repasaremos tan solo las particularidades de la *blockchain* de Ethereum, dado que el funcionamiento de la *blockchain* de forma general ya ha sido descrito en otros módulos.

En el apartado 3.4 veremos cuáles son las principales redes de Ethereum, tanto públicas como privadas.

Finalmente, en el apartado 3.5 describiremos otros protocolos que conforman el ecosistema de Ethereum y que, de forma conjunta, nos permiten desarrollar aplicaciones descentralizadas completas.

3.1. Clientes

En Ethereum, los participantes de la red se instalan un software que se conoce como *cliente*. Dependiendo de las opciones con las que se despliegue un cliente, este podrá ejercer diferentes funciones dentro de la red.

Además, al tratarse de un proyecto *open source*, existe la posibilidad de que cualquier programador implemente un cliente que siga el protocolo de Ethereum. Por este motivo, en Ethereum existen diferentes clientes desarrollados por diferentes equipos de trabajo y que utilizan distintos lenguajes de programación. Esto es positivo en el sentido de que si existe una pluralidad de clientes, se evita que siempre se utilice el mismo producto y depender en exceso de él, lo que ayuda a que, ante un fallo o vulnerabilidad, no todos los nodos se vean afectados y la red pueda continuar funcionando correctamente y de forma segura. Algunos de los clientes que se encuentran en activo actualmente son: Geth, OpenEthereum (antiguo Parity) y Trinity, cada uno desarrollado en un lenguaje de programación diferente: Go, Rust y Python, respectivamente.

Los nodos de la red de Ethereum pueden ejecutar tres tipos de cliente distintos en función de los datos que procesan, lo que da lugar a los siguientes tipos de nodo:

1) Nodo completo. Nodo que se encarga de la validación y verificación de bloques, además se encarga de la verificación de los estados. Almacena datos completos de la cadena de bloques de Ethereum, por lo que puede servir toda la información de la red, por este motivo también es capaz de proporcionar datos bajo demanda. Es importante remarcar que los nodos completos almacenan los encabezados y contenidos de bloque indefinidamente, pero no almacenan el estado global de todos los bloques desde su origen, de esto se encarga el nodo de archivo.

2) Nodo ligero. Nodo que únicamente almacena los *block headers* y solicita el resto de la información a algún nodo completo, o de archivo en caso de ser necesario. Es capaz de verificar la validez de los datos que recibe al realizar una comparación con el estado en ese momento, representado por «eth1 State» en

la figura 2, de los *block headers* que tiene almacenados acerca de estos datos, que es el *hash* obtenido a partir de que todas las transacciones se hayan ejecutado y finalizado. Es común su utilización en dispositivos con baja capacidad computacional y que están limitados en cuanto al almacenamiento, como teléfonos móviles.

3) Nodo de archivo. Nodo que almacena todo aquello que se guarda en el nodo completo. Es capaz de construir un archivo de estados históricos de la red. Puesto que precisan de un elevado almacenamiento de información, este tipo de nodos no es utilizado por los usuarios, sino por servicios como exploradores de bloques o *wallets*.

Así, para interactuar con Ethereum es necesario tener alguno de los clientes instalados. Además, también es necesario definir una entidad que represente los contratos y los balances de los diferentes usuarios. En Bitcoin esta entidad son las direcciones, que, como sabemos, se derivan de una clave pública y pueden controlarse con la clave privada asociada. Ethereum funciona de forma parecida, pero, en este caso, esta entidad se denomina *cuenta* y puede representar tanto a un usuario como a un contrato. En el siguiente apartado detallamos cómo funcionan las cuentas en Ethereum.

3.1.1. Cuentas de Ethereum

Una cuenta dentro de Ethereum es una entidad que consta de un balance de *ether* y que puede enviar transacciones en la red de Ethereum. Existen distintos tipos de cuentas, las que pueden ser controladas por usuarios o las ejecutadas como contratos inteligentes.

- **Cuentas de propiedad externa**: también conocidas como EOA (*externally owned account*). Se trata de cuentas externas controladas por pares de claves público-privadas, y que son controladas por humanos. Estas cuentas son similares a las direcciones de Bitcoin, que se pueden crear de forma autónoma por parte de los usuarios sin ningún coste. Entre las cuentas de este tipo, las únicas transacciones permitidas son las transferencias de ETH. No obstante, la interacción de una cuenta de propiedad externa con una cuenta de contrato sí es posible, y viceversa.

- **Cuentas de contrato**: Se trata de cuentas de contratos inteligentes implementados en la red y que son generadas a la hora del despliegue del contrato inteligente. A diferencia de las cuentas de propiedad externa, estas tienen un coste al ser creadas por usar almacenamiento de la red (el código del contrato inteligente que se guarda en la cadena de bloques) y no están limitadas a la transferencia de ETH, sino que permiten acciones implementadas en el código del contrato, así como transferencia de *tokens* o creación de nuevos contratos.

Una diferencia clave entre ambas cuentas es que las cuentas de propiedad externa pueden iniciar transacciones, mientras que las cuentas de contrato solo pueden enviar una transacción en respuesta a una transacción recibida.

Es importante destacar que ambos tipos de cuentas comparten el mismo espacio de direcciones en Ethereum y son tratados por igual por la EVM. De hecho, ambas tienen la misma representación y, a simple vista, no se puede determinar si una cuenta pertenece a un contrato o es de propiedad externa.

Ejemplo de cuenta en Ethereum:

```
0x5e97870f263700f46aa00d967821199b9bc5a120
```

Tabla 1. Elementos de las cuentas de Ethereum

nonce	Contador que indica el número de transacciones enviadas para cuentas de propiedad externa. Para cuentas de contrato, este contador indica el número de contratos inteligentes creados por la cuenta.
balance	Valor que representa el número de wei (unidad mínima de denominación del *ether*, en 1 ETH hay 1e + 18 wei) que posee esta cuenta. Más información sobre la criptomoneda ETH en la sección 4.2.
code hash	*Hash* (Keccak 256) del código de la EVM, o *bytecode* de esta cuenta, es decir, este es el *hash* del código que se ejecutará. Al contrario que el resto de elementos de la cuenta, es inmutable y no se puede modificar. Para cuentas de propiedad externa este elemento es el *hash* de un *string* vacío.
storage root	*Hash* (Keccak 256) del nodo raíz de un árbol Merkle Patricia que codifica el contenido del almacenamiento de la cuenta.

Fuente: elaboración propia

3.2. Máquina virtual de Ethereum

La máquina virtual de Ethereum (*ethereum virtual machine*, o EVM por sus siglas) es el entorno de ejecución de contratos inteligentes de Ethereum, es decir, es quien se encarga del despliegue y ejecución de los contratos inteligentes. Es importante destacar que cuando sucede una transacción entre cuentas EOA para la transferencia de *ether*, la EVM no se involucra en la ejecución, y la transferencia de *ether* sucede directamente.

Como se ha comentado antes, una diferencia fundamental entre Bitcoin y Ethereum es que el lenguaje de *scripting* de Bitcoin se ejecuta secuencialmente y no acepta bucles y, en cambio, en Ethereum sí que es posible la creación de bucles. De hecho, más concretamente, la EVM se considera una máquina de estado cuasi Turing completa (*quasi Touring complete*) debido a que, aunque se pueden crear bucles, la ejecución dentro de la EVM está limitada a un número finito de pasos computacionales para evitar que algún usuario malintencio-

nado, o que haya cometido algún error de programación, ejecute contratos inteligentes con bucles infinitos o demasiado largos que cuelguen la ejecución en los nodos. Esta limitación se ejerce mediante el uso del gas.

El gas es la unidad en la que se mide la cantidad de esfuerzo computacional que se requiere en Ethereum para la ejecución de operaciones. Cada transacción en Ethereum consta de una serie de operaciones que están asociadas a un coste computacional, por lo tanto, la ejecución de la transacción vendrá determinada por una cierta cantidad de gas. Este gas se puede considerar como una tarifa que hay que pagar para que la transacción se ejecute exitosamente. Estas tarifas de gas vienen determinadas en *ether*, concretamente en Gwei (0.000000001 ETH), por lo que se pagan con el *token* nativo de Ethereum.

Por lo tanto, para la ejecución de cualquier contrato inteligente será necesaria una cantidad de gas. Es importante tener en cuenta que el gas no es un recurso infinito, y que dentro de Ethereum, cada uno de los bloques tiene un parámetro que fija la cantidad de gas máximo a gastar en ese bloque. De esta forma, este límite actúa como mecanismo de protección frente a ejecuciones que puedan quedar bloqueadas o en bucle, y son rechazadas en caso de superar este límite. Por otro lado, para la ejecución de cualquier contrato inteligente será necesaria una cantidad de gas. La ejecución se interrumpirá en el caso de que un minero ejecute un número suficiente de operaciones que agoten el gas asignado por quien ha iniciado la transacción. No debe confundirse el gas límite que tiene el bloque con el gas límite que fija un usuario al enviar una transacción, aunque en caso de agotarse el gas en ambos casos la respuesta será que la ejecución se detenga. Estos mecanismos hacen que operaciones como bucles infinitos no sean realizables dentro de la EVM.

Cuando la EVM ejecuta código de un contrato inteligente, sucede lo que se conoce como un *cambio de estado*, que viene determinado por una transacción. Esto quiere decir que Ethereum antes de la ejecución de la transacción tenía un estado «X», y al finalizar la transacción tendrá un estado «X + 1». El estado es un mapeo entre cuentas, y el valor que hay en sus elementos: *nonce*, *balance, code hash* y *storage root*. Todo esto puede verse como un estado global del sistema que se actualiza constantemente mediante la ejecución de transacciones.

Por ejemplo, si hay un contrato inteligente que almacena los nombres de todos los asistentes a una reunión, cada vez que se agrega un nuevo integrante a la reunión a partir de una nueva transacción, el estado del contrato inteligente se modificará.

La EVM consta de los siguientes componentes de datos:

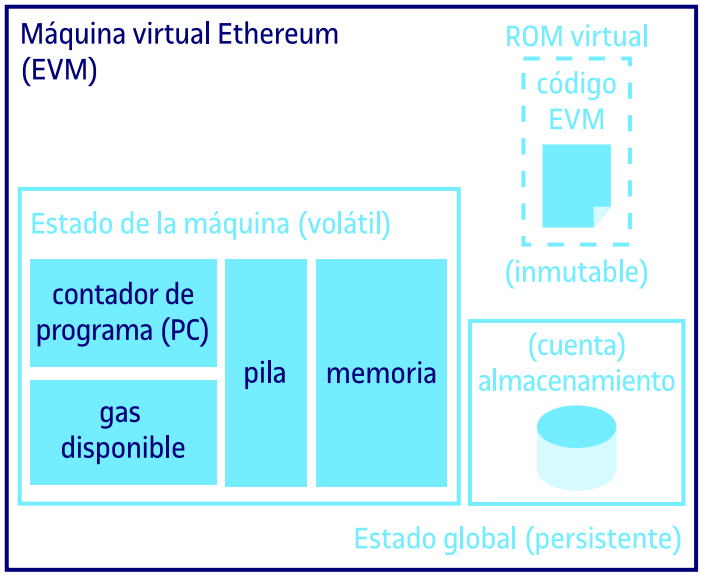
- Una memoria ROM virtual, que cargará el *bytecode* de los contratos que se ejecutarán.
- Una memoria volátil inicializada a cero para almacenar datos de forma temporal.

- Un almacenamiento permanente inicializado a cero, que forma parte del estado de Ethereum.

La EVM tiene una arquitectura basada en pila, con una profundidad de 1.024 elementos que vienen definidos como palabras de 256 bits. Este tamaño de palabra se definió basándose en la función *hash* SHA3-256 y para facilitar las operaciones con curvas elípticas, es decir, para que tuviese el mismo tamaño de palabra que dicha función *hash*. A continuación, ampliaremos la información sobre cómo se genera el *bytecode* de un contrato inteligente para que pueda ser interpretado por la EVM.

Otro elemento que puede observarse en la figura 3 y que forma parte de la EVM es el contador de programa utilizado para indicar la posición de la EVM durante la ejecución de instrucciones. Además, la EVM contiene el gas disponible para determinar si al agotarse el gas debe finalizar la ejecución con la excepción de que el gas disponible se haya agotado.

Figura 3. Representación de la EVM



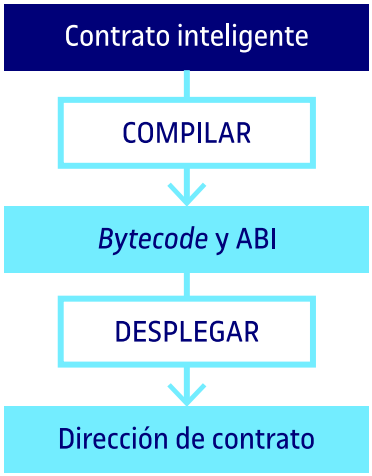
Fuente: https://ethereum.org/en/developers/docs/evm/

3.2.1. El *bytecode* de la EVM

Los contratos inteligentes típicamente se desarrollan en Solidity, aunque también se pueden desarrollar en Vyper, que es otro de los lenguajes de programación de contratos inteligentes más activo y mantenido. Aquellos desarrolladores con más experiencia y con un conocimiento más avanzado pueden utilizar el lenguaje Yul.

Cuando se desarrolla un contrato inteligente y se compila, se obtiene lo que se conoce como el *bytecode*. El *bytecode* se define como un conjunto de instrucciones diseñado para la ejecución por parte de un intérprete de software o una máquina virtual. A diferencia del código fuente legible por humanos, el *bytecode* se expresa en formato numérico.

Figura 4. Proceso de compilación y despliegue de un contrato inteligente



Fuente: elaboración propia

Este *bytecode* del contrato inteligente ya compilado se desplegará y ejecutará en la EVM como una serie de instrucciones definidas como códigos de operación. Veamos a modo de ejemplo alguno de los códigos de operación más utilizados por la EVM.

- ADD: operador aritmético que realiza la suma de dos elementos de la pila. Su coste es de 3 unidades de gas.
- GASPRICE: obtiene el precio del gas en el entorno actual. Su coste es de 2 unidades de gas.
- BALANCE: obtiene el saldo de la cuenta dada. Su coste es de 700 unidades de gas.

Además, la EVM es capaz de manejar excepciones que pueden suceder por distintos motivos, como por ejemplo no tener suficiente gas para realizar la instrucción o intentar ejecutar instrucciones no válidas terminando la máquina inmediatamente frente a estas situaciones.

A continuación se muestra el código de un contrato inteligente desarrollado en Solidity:

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

contract SimpleStorage {
```

```
 uint storedData;

 function set(uint x) public {
      storedData = x;
 }

 function get() public view returns (uint) {
      return storedData;
 }
}
```

Una vez se ha compilado el código del contrato inteligente anterior, el compilador, en este caso de Solidity, nos devuelve el *bytecode* que ejecutará la EVM. Este *bytecode* normalmente viene representado en hexadecimal, aunque también puede estar representado mediante los códigos de operación de la EVM.

Bytecode:

```
608060405234801560f57600080fd5b5060ac8061001e6000396000f3fe6080604052348015600f57600080fd
5b506004361060325760003560e01c806360fe47b11460375780636d4ce63c146053575b600080fd5b60516004
803603602081101560604b57600080fd5b5035606b565b005b60596070565b604080519182525190819003602001
90f35b600055565b6000549056fea2646970667358221220086b068830652f463f779dc171f02bdd89688f359a
d7052d70f0ec3c632be36464736f6c63430007060033
```

Instrucciones o códigos de operación:

```
PUSH1 0x80 PUSH1 0x40 MSTORE CALLVALUE DUP1 ISZERO PUSH1 0xF JUMPI PUSH1 0x0 DUP1 REVERT
 JUMPDEST POP PUSH1 0xAC DUP1 PUSH2 0x1E PUSH1 0x0 CODECOPY PUSH1 0x0 RETURN INVALID PUSH1
0x80 PUSH1 0x40 MSTORE CALLVALUE DUP1 ISZERO PUSH1 0xF JUMPI PUSH1 0x0 DUP1 REVERT JUMPDEST
POP PUSH1 0x4 CALLDATASIZE LT PUSH1 0x32 JUMPI PUSH1 0x0 CALLDATALOAD PUSH1 0xE0 SHR DUP1
PUSH4 0x60FE47B1 EQ PUSH1 0x37 JUMPI DUP1 PUSH4 0x6D4CE63C EQ PUSH1 0x53 JUMPI JUMPDEST
PUSH1 0x0 DUP1 REVERT JUMPDEST PUSH1 0x51 PUSH1 0x4 DUP1 CALLDATASIZE SUB PUSH1 0x20 DUP2
LT ISZERO PUSH1 0x4B JUMPI PUSH1 0x0 DUP1 REVERT JUMPDEST POP CALLDATALOAD PUSH1 0x6B JUMP
JUMPDEST STOP JUMPDEST PUSH1 0x59 PUSH1 0x70 JUMP JUMPDEST PUSH1 0x40 DUP1 MLOAD SWAP2 DUP3
MSTORE MLOAD SWAP1 DUP2 SWAP1 SUB PUSH1 0x20 ADD SWAP1 RETURN JUMPDEST PUSH1 0x0 SSTORE JUMP
JUMPDEST PUSH1 0x0 SLOAD SWAP1 JUMP INVALID LOG2 PUSH5 0x6970667358 0x22 SLT KECCAK256 ADDMOD
PUSH12 0x68830652F463F779DC171F0 0x2B 0xDD DUP10 PUSH9 0x8F359AD7052D70F0EC EXTCODECOPY PUSH4
0x2BE36464 PUSH20 0x6F6C6343000706003300000000000000000000000
```

3.3. Protocolo de consenso en Ethereum

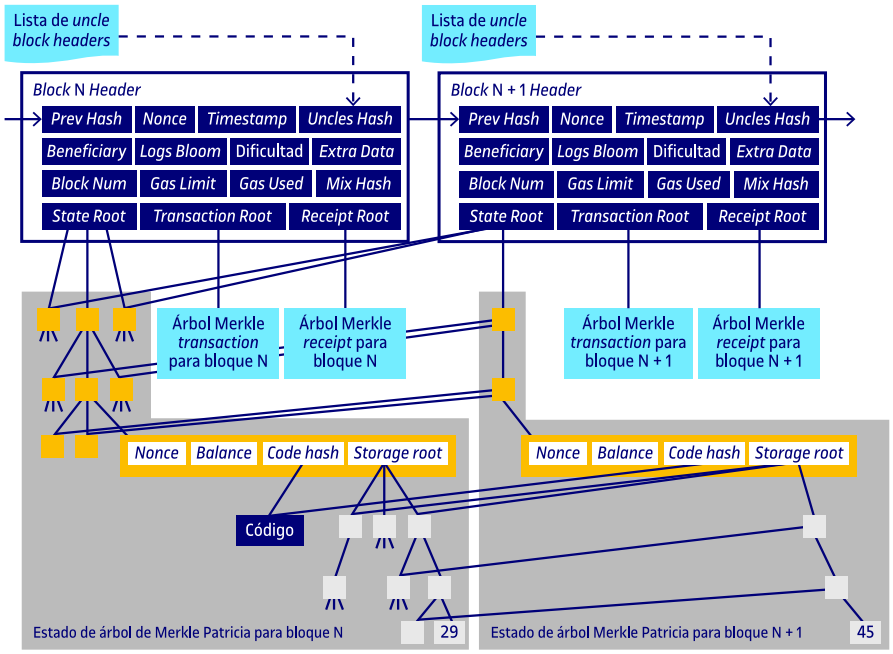
Al igual que en Bitcoin y otros protocolos, Ethereum 1.0 utiliza el algoritmo de consenso de prueba de trabajo (PoW) persiguiendo el mismo objetivo, es decir, asegurar la cadena de bloques mientras la red se mantiene distribuida y descentralizada. En cambio, Ethereum 2.0 está pensado para utilizar el algoritmo de consenso de prueba de participación, que ha sido bautizado como *Casper*.

La particularidad de Ethereum es que, además de permitir transacciones de valor como en otras *blockchains*, permite la ejecución de contratos inteligentes. Estos contratos inteligentes tienen un estado, el cual se actualiza en función de las interacciones de los usuarios con el contrato. Estas interacciones, que se reflejan mediante transacciones, realizan modificaciones sobre el contrato, por ejemplo, en las variables que almacena. La cadena de bloques de Ethereum, al guardar el estado de todas las cuentas, tanto EOA como cuentas de contrato, almacenará en la cadena de bloques el estado en el que se encuentren los contratos inteligentes, es decir, almacena el valor de las variables. Además, en las cuentas de contrato se almacena el código compilado del contrato inteligente (*bytecode*), por lo tanto el contrato inteligente está también almacenado en la cadena de bloques y todos los nodos tienen acceso a él.

Aunque el algoritmo de PoW que utiliza Ethereum comparte la misma idea que el de Bitcoin u otros protocolos, tiene algunas diferencias al respecto de estos en cuanto a funcionalidad y diseño, este algoritmo es conocido como *Ethash*.

Haciendo referencia a lo comentado anteriormente acerca del almacenamiento de la información en la *blockchain* de Ethereum, dentro de los bloques existen una serie de elementos que almacenan la información del estado de las cuentas, tanto cuentas EAO como cuentas de contratos inteligentes.

Figura 5. Estructura de los bloques y representación del estado de Ethereum



Fuente: https://ethereum.stackexchange.com/questions/268/ethereum-block-architecture/757#757

En la figura 5 se muestra una visión global de la estructura de un bloque de la *blockchain* de Ethereum. A partir de esta imagen se mostrará cómo la información se almacena en Ethereum, concretamente el estado de las cuentas, y cómo la variación de información en un contrato inteligente o en el balance de una cuenta se almacena utilizando los árboles de Merkle Patricia.

Los árboles de Merkle Patricia son las estructuras de datos en forma de árbol utilizadas en Ethereum para almacenar la información. La utilización de este tipo de estructura se debe a que el estado en Ethereum se actualiza frecuentemente, con lo cual, una estructura como el árbol de Merkle binario tendría que ser recalculado constantemente; en cambio, utilizando los árboles Merkle Patricia no es necesario recalcular toda la estructura (o árbol) para obtener la raíz después de que nuevos datos se hayan añadido, modificado o borrado.

Como se puede observar en la figura 5, en la parte inferior de la imagen, cuando un dato es modificado y, por ejemplo, pasa de valer 29 a 45, esta nueva información se actualiza y forma un nuevo nodo en el árbol, por lo que se olvida el nodo del valor anterior. Sin embargo, se puede ver que el resto de la información hace referencia al árbol del bloque anterior debido a que no ha sufrido ninguna modificación.

En Ethereum existen cuatro árboles, que vienen referenciados en el bloque a partir de sus raíces:

- *Receipt* (‘recibo de la transacción’, en español), cuya raíz viene referenciada por el parámetro *receipt root*.
- *Transaction* (‘transacción’, en español), cuya raíz viene referenciada por el parámetro *transaction root*.

- *State* ('estado', en español), cuya raíz viene referenciada por el parámetro *state root*.
- *Storage* ('almacenamiento', en español), cuya raíz viene referenciada por el parámetro *storage root*.

En el caso de los árboles que se construyen para los elementos *receipt* y *transaction*, son independientes y únicos para cada uno de los bloques de la cadena, debido a que se construyen a partir de las transacciones que componen un bloque. El árbol *transaction*, tal y como su nombre indica, registra las transacciones; mientras que el árbol *receipt* registra el resultado de las mismas y muestra información básica de la ejecución de las transacciones, de los eventos producidos, del coste, *hash* de la transacción, etc.

En cambio, el árbol que constituye el estado global de Ethereum, conocido como *state*, se actualiza según avanza la cadena de bloques y, como se puede observar en la figura 5, se compone de nodos (cuadrados amarillos) que hacen referencia a cuentas de Ethereum. A esta información sobre el estado en Ethereum se puede acceder como en un mapa clave-valor, en el cual las claves son direcciones de las cuentas (representadas en la figura por los cuadrados amarillos), y los valores son los elementos que componen una cuenta: *nonce*, *balance*, *code hash* y *storage root* (representados en la figura por los cuadrados blancos dentro de cada cuadro amarillo y explicados en el apartado 3.1.2). A su vez, el *storage root* es una raíz de un árbol de Merkle Patricia (*storage*) que se construye a partir del contenido almacenado en una cuenta.

A partir de lo anterior y utilizando una ejemplificación, si se despliega en Ethereum un contrato inteligente que almacena el valor de una variable, el código del contrato será almacenado en el *code hash* de la cuenta, que es un elemento de la misma. La variable del contrato inteligente, inicialmente con valor 29, formará parte del árbol de almacenamiento e influirá en el valor del *storage root*. La clave del mapa clave-valor del estado que identifica a estos elementos es la dirección del contrato dentro de Ethereum, que se obtiene una vez ha sido desplegado.

Si posteriormente la variable del contrato se modifica, se producirá lo que se conoce como un *cambio de estado*. Este cambio de estado viene determinado por el nuevo valor de la variable, que en el ejemplo de la figura 5 es 45. La raíz del árbol de almacenamiento será diferente porque al actualizar el valor de la variable, el nodo con valor 29 se olvida en favor de un nuevo nodo con valor 45. A partir de este nuevo valor se construye la nueva raíz del árbol que se actualizará en el elemento *storage root* de la cuenta, pero como se puede observar en la imagen, el resto de información es referenciada respecto del bloque anterior, por lo que la construcción de la nueva raíz será mucho más eficiente que si se tuviese que recalcular desde cero. Asimismo, como el código del contrato almacenado en el elemento *code hash* no puede ser modificado, también se referencia a partir del bloque anterior, que es donde se desplegó el

contrato. Con respecto al *nonce* y al *balance*, el funcionamiento es exactamente el mismo, si se modifica se crea un nuevo nodo en el árbol, y si es igual, se hace referencia al nodo del árbol del bloque anterior.

A partir de todo lo anterior, se puede obtener una visión global de cómo la información es estructurada y almacenada dentro de la *blockchain* de Ethereum.

3.3.1. Ethereum 1.0: Ethash

Ethash es el algoritmo de consenso de prueba de trabajo utilizado por el sistema Ethereum. En cuanto a su funcionamiento, los mineros realizan un proceso de prueba y error compitiendo entre ellos en búsqueda del *nonce* para el siguiente bloque, al igual que sucede en Bitcoin. En cambio, a diferencia del algoritmo utilizado por Bitcoin, que se centra en variar el parámetro *nonce* y computar el *hash* del encabezado de bloque tantas veces como sea necesario hasta que resulte un número menor que un parámetro dado por la red (el parámetro Dificultad); en el algoritmo utilizado por Ethereum se utilizan otros parámetros más allá de los incluidos en la cabecera del bloque para añadir un mayor coste computacional. El uso de estos nuevos parámetros hace que el algoritmo de consenso de Ethereum pueda resolverse de forma eficiente mediante GPU, en lugar de CPU, como sucede en Bitcoin y otros algoritmos de prueba de trabajo. Este es un hecho interesante debido a que hace más accesible el minado. En protocolos de consenso como el de Bitcoin, los mineros utilizan equipos de circuito integrado de aplicación específica (ASIC, del inglés *application-specific integrated circuit*) diseñados específicamente para resolver los problemas criptográficos del minado, con lo que intentar competir contra estos mineros utilizando un ordenador de sobremesa es un intento fútil. Sin embargo, el uso de GPU es más económico y está muy extendido.

Estos nuevos parámetros de Ethash son un conjunto de datos que forman un DAG (gráfico acíclico dirigido), que inicialmente comenzó siendo de 1GB, pero cuyo tamaño va aumentando de forma lineal con el paso del tiempo y se va actualizando cada 30.000 bloques. Este conjunto de datos depende de una semilla, que se obtiene a partir de los bloques generados hasta el momento y que, además, se utiliza para construir una caché que será utilizada para generar el DAG. Además, la caché también será utilizada para la verificación por otros clientes y mineros, por lo que no es necesario que se almacene el DAG, sino solamente esta caché.

El proceso de minería consiste en tomar de forma aleatoria pequeños fragmentos del DAG y combinarlos con los datos que forman el encabezado de bloque (raíz de Merkle de las transacciones, Dificultad, *hash* del bloque anterior, límite de gas, etc.) junto con el *nonce*. Una vez combinados, se va modificando el valor del *nonce* hasta que se da con uno en el cual el *hash* de la combinación anterior está por debajo de un objetivo definido por el parámetro Dificultad. Este parámetro viene determinado por la red, y se calcula para imponer una dificultad de minado a cada bloque para mantener una distancia regular entre

bloque y bloque. Hay que tener en cuenta que si este parámetro no cambiara con el tiempo, las mejoras en el hardware permitirían cada vez minar bloques más rápidamente.

La verificación por parte del resto de nodos se puede realizar a partir de regenerar pequeñas partes del DAG usando la caché almacenada. Esto hace posible que el proceso de verificación se ejecute en poco tiempo, además de utilizar muy poca memoria debido a que la caché tiene un tamaño de 16 MB, lo que evita almacenar grandes cantidades de datos y habilita a clientes ligeros el almacenamiento de la misma.

En cuanto al funcionamiento, como se puede observar, es similar al utilizado por Bitcoin, en el sentido de que es un proceso iterativo que busca un *nonce* para obtener un *hash* por debajo de un objetivo, pero que utiliza otro tipo de parámetros en su proceso de minería para habilitar que se pueda realizar el minado con GPU.

3.3.2. Ethereum 2.0

Ethereum 2.0, también llamado Eth2, o Serenity, es la próxima actualización de la cadena de bloques Ethereum (ConsenSys, 2020 y Muzzy E, 2020). Ethereum 2.0 será desplegado en múltiples fases (EthHub, 2020). La Fase 0 se inició en 2020 y, posteriormente, cada una de las fases mejorará la funcionalidad y rendimiento de Ethereum de distintas formas, aumentando la velocidad, eficiencia y escalabilidad, lo que permitirá aumentar el número de transacciones por segundo. En Ethereum 1.0, todo esto son problemas importantes para las aplicaciones desarrolladas, ya que en momentos de alta demanda, la red se satura y provoca que las transacciones tarden mucho en incluirse en la *blockchain* y las comisiones pueden resultar extremadamente elevadas, ya que pueden superar los 50 dólares por cada transacción.

Ethereum 2.0 supone cambios fundamentales en el diseño y la estructura respecto de su versión 1.0. Los cambios principales son la implantación de la prueba de participación y las *shard chains*.

- **Prueba de participación**: actualmente Ethereum 1.0 utiliza el algoritmo de consenso conocido como *prueba de trabajo* (PoW). La implantación de la prueba de participación (en inglés, *Proof-of-Stake*, o PoS) se trata de una actualización que permite mejorar la seguridad, la escalabilidad y la eficiencia energética. En PoS, los nodos que validan las transacciones y crean los nuevos bloques de la cadena deben dejar un depósito en criptomonedas antes de poder proponer un bloque nuevo. En el caso de proponer un bloque incorrecto, dicho nodo perdería su depósito. Es importante destacar que este sistema para incentivar la creación correcta de bloques y el consenso entre todos los nodos de la red ya no requiere de la resolución de puzles criptográficos complejos y de un alto consumo energético como pasaba en PoW. La prueba de participación se basa en validadores y en sus

respectivos depósitos de *ether*, en lugar de en la potencia de cómputo de los mineros físicos.

- ***Shard chains***: este tipo de cadenas son un mecanismo de escalabilidad que busca mejorar el rendimiento de Ethereum. Actualmente, Ethereum tiene una sola cadena formada por bloques consecutivos que ofrece una alta seguridad y facilidad de verificación de la información. El hecho de procesar y validar cada transacción siguiendo esta estructura de bloques consecutivos afecta a la eficiencia en el procesamiento de transacciones, lo que provoca en ocasiones una congestión y saturación de transacciones en la red de Ethereum. Las *shard chains* son el mecanismo mediante el cual la cadena de bloques de Ethereum se divide, lo que hace que la responsabilidad del procesamiento de la información también se divida entre varios nodos. Este tipo de cadenas permiten que las transacciones se procesen en paralelo, en lugar de consecutivamente, y por tanto, un mayor número de cadenas en paralelo conllevará un rendimiento mucho mayor.

Esta actualización no supone que la cadena de Ethereum 1.0 vaya a dejar de existir, de hecho el plan actual es que la cadena Ethereum 1.0 se convierta en la primera *shard chain* de Ethereum 2.0 a partir de la fase 1. Ethereum 1.0 seguirá actualizándose y su desarrollo no se abandonará incluso mientras se desarrolla Ethereum 2.0.

El plan de programación propuesto por Ethereum consta de las siguientes fases:

- **Fase 0**: en ella se implementará la Beacon Chain. Esta Beacon Chain se encargará de almacenar y gestionar el registro de validadores y también implementará el mecanismo de consenso de PoS para Ethereum 2.0. Asimismo, Ethereum 1.0 continuará funcionando de manera normal junto con la nueva Beacon Chain, lo que asegurará que no haya interrupciones en la continuidad de los datos.

- **Fase 1**: lanzada durante el año 2021. La mejora principal de esta fase es la integración de las *shard chains* para mejorar la escalabilidad de Ethereum haciendo una división en 64 cadenas distintas. La estimación más conservadora es que permita 64 veces más rendimiento que Ethereum 1.0, aunque el diseño está pensado para lograr un rendimiento cientos de veces mayor.

- **Fase 2**: planificada originalmente entre 2021 y 2022. Esta es la fase que menos definida se encuentra actualmente en comparación con las otras dos fases. En ella, se agregarán cuentas de *ether* para permitir transferencias, se implementarán transferencias entre *shard chains* y contratos y, finalmente, se llevará Ethereum 1.0 a Ethereum 2.0 y se eliminará definitivamente la prueba de trabajo.

3.4. Redes de Ethereum

Al ser Ethereum un protocolo, pueden existir distintas redes independientes entre sí, pero que están basadas en dicho protocolo. Una red de Ethereum, por lo tanto, se puede definir como un entorno en el cual está implementado su protocolo, que cubre una necesidad y está enfocada a un caso de uso determinado, como pueden ser el de producción o el de prueba.

3.4.1. Redes públicas

Las redes públicas son aquellas accesibles para cualquier persona que disponga de una conexión a Internet. No se hace una discriminación de usuarios en estas redes, cualquiera puede desplegar un nodo y crear transacciones e inspeccionar la *blockchain*, y dependiendo de la red concreta, también podrá validar transacciones de otros usuarios y participar en el protocolo de consenso. Es importante distinguir entre dos tipos de redes públicas:

- **Red principal**: también conocida como *Mainnet*, se trata de la red pública principal de Ethereum, en la cual ocurren las transacciones, las cuales tienen un coste real en *ether* y, por lo tanto, monetario. Esta es la red en la que todo lo que sucede y se realiza tiene un impacto real.

- **Redes de prueba**: son aquellas redes sin impacto real, pero que funcionan de forma parecida a la red principal con el fin de ofrecer un entorno de prueba. En estas redes, el *ether* utilizado no tiene coste monetario real. Este tipo de redes son comúnmente utilizadas por desarrolladores de contratos inteligentes, aplicaciones descentralizadas u otros protocolos, y sirven para probar el funcionamiento de sus desarrollos en entornos similares o los más parecidos a la red principal. Al contrario que la red principal, que es única, dentro de las redes de prueba encontramos las siguientes: Goerli, Kovan, Rinkeby y Ropsten. Entre las diferentes redes de prueba existen algunas diferencias, como por ejemplo el protocolo de consenso concreto usado en cada una de ellas.

3.4.2. Redes privadas

Una red privada de Ethereum es aquella en la cual los nodos no están conectados a las redes públicas mencionadas anteriormente, es decir, la red principal o redes de prueba. Las redes privadas reciben este nombre porque se encuentran aisladas y, en ocasiones, suelen estar protegidas. Dentro de las redes privadas, existen dos modelos básicos, cada uno destinado a una finalidad distinta.

Por una parte, están las redes destinadas al desarrollo y prueba de aplicaciones o contratos inteligentes. Para no depender de una red de prueba pública, los desarrolladores de aplicaciones utilizan redes en modo local, donde se levanta una instancia de una cadena de bloques en la que el usuario puede probar su

desarrollo y el comportamiento de su aplicación. Estas redes son muy utilizadas en etapas tempranas del desarrollo por su rápida interacción y facilidad de uso.

Por otra parte, existen redes privadas o consorcios, que son aquellas formadas por agrupaciones de instituciones (empresas, administración pública, etc.) en la cual todos los nodos son conocidos y, por lo tanto, existe una confianza en ellos, donde la validación de transacciones se realiza por unos nodos prefijados por este grupo de instituciones. El acceso a esta red está restringido a los nodos pertenecientes a las instituciones que forman parte de la red. Quorum y Hyperledger Besu son los productos más conocidos para utilizar Ethereum como una red privada de este tipo.

3.5. Ecosistema de Ethereum

Hasta este punto hemos descrito los componentes básicos de la arquitectura de Ethereum. Con Ethereum disponemos de una especie de computadora mundial que guarda de forma distribuida un estado global para todas sus aplicaciones. Sin embargo, esto no es suficiente para poder responder a todas las necesidades de las aplicaciones descentralizadas. Por esto se han desarrollado otros muchos protocolos que, junto con Ethereum, sirven de base para desplegar cualquier tipo de aplicación. En el apartado 3.5.1 describimos Swarm, un protocolo de almacenamiento distribuido; y en el 3.5.2 describimos ENS, un servicio de nombres para Ethereum.

Además, las aplicaciones descentralizadas necesitan a veces información que no está en la *blockchain*. Estas fuentes de información se conocen como *oráculos* y los describimos en el apartado 3.5.3.

Finalmente, durante su existencia, Ethereum ha presentado problemas de escalabilidad que provocan que las transacciones no se registren en la *blockchain* lo suficientemente rápido. Esto hace que en muchas ocasiones se requieran mecanismos adicionales para poder interactuar con la *blockchain* de forma que sea efectiva para las aplicaciones descentralizadas. En el apartado 3.5.4 describimos lo que se conoce como *soluciones de segunda capa*, sistemas que ofrecen funcionalidades adicionales o sistemas que se basan en Ethereum para guardar el estado global, pero que utilizan ciertas técnicas para no verse afectados por los problemas de escalabilidad.

3.5.1. Swarm

Swarm (también conocido como BZZ) es un protocolo de almacenamiento distribuido y un servicio de distribución de contenido. Swarm es una red de almacenamiento descentralizada, distribuida y *peer-to-peer*. Los archivos en es-

ta red se identifican mediante el *hash* de su contenido. Swarm se desarrolla como un servicio nativo de capa base para la pila Ethereum Web 3.0. Swarm está integrado con devp2p, que es la capa de red multiprotocolo de Ethereum.

El objetivo de Swarm es disponer de una solución de almacenamiento y servicio *peer-to-peer* que no tenga tiempo de inactividad (es decir, que esté siempre disponible), que sea resistente a los ataques de denegación de servicio distribuidos (DDoS), tolerante a fallos y resistente a la censura, así como autosostenible gracias a un sistema de incentivos integrado. Dicho sistema de incentivos modela a los participantes individuales como agentes que siguen su propio interés racional; de esta forma, aunque no exista un nodo coordinador, el comportamiento global de la red es beneficioso para todos los participantes.

A diferencia de sistemas descentralizados de almacenamiento distribuido, como puede ser BitTorrent, donde el contenido se aloja en un *host* y se comparte desde este mismo servidor, Swarm también permite que cualquier nodo pueda simplemente cargar contenido en Swarm y luego desconectarse. BitTorrent es capaz de entregar grandes cantidades de datos con alta latencia, pero Swarm, además, es capaz de entregar pequeños fragmentos de datos con baja latencia.

Otra diferencia fundamental en comparación con otros sistemas descentralizados actuales es el sistema de incentivos que se ha introducido previamente. Este sistema permite a los participantes agrupar sus recursos de almacenamiento y ancho de banda con el fin de proporcionar servicios de contenido global a todos los participantes, al igual que sucede en otros protocolos como BitTorrent, pero con la diferencia de que los usuarios son incentivados por aportar sus recursos de almacenamiento y ancho de banda a la red. Así, los usuarios son recompensados por entregar archivos solicitados o, incluso, por almacenar archivos poco solicitados durante un largo tiempo, además de poder fijar unas tasas de solicitud y envío de información mediante las cuales se puede determinar el precio que se va a pedir por la solicitud de un archivo y el precio que se va a pedir por enviarlo utilizando su ancho de banda.

Un par de casos de uso representativos de Swarm son: el alojamiento de aplicaciones web dinámicas que son visitadas en tiempo real y requieren baja latencia, y el almacenamiento de datos estáticos de gran volumen que no requieren baja latencia pero sí un gran ancho de banda para poder ser descargados.

3.5.2. Ethereum Name Service

Ethereum Name Service ('servicio de nombres de Ethereum', en español), o ENS, es un sistema de nombres distribuido, abierto y extensible basado en la cadena de bloques de Ethereum.

ENS pretende atajar un problema parecido al que ataja el sistema de nombres de dominio (*domain name system*, o DNS) para Internet, aunque los detalles técnicos y de implementación son sustancialmente diferentes. Co-

mo es bien sabido, DNS es un sistema que se encarga de traducir los nombres de dominio web, como www.google.com, a una dirección IP como 10.128.1.253. De una forma parecida, ENS se encarga de traducir identificadores legibles por máquinas, como una dirección de Ethereum a nombres legibles por los humanos. Por ejemplo, en estos momentos, la dirección «0xd1220a0cf47c7b9be7a2e6ba89f429762e7b9adb» está registrada por el dominio «alex.vandesande.eth». De esta forma, en vez de tener que recordar una dirección de Ethereum, podemos tan solo recordar el dominio, de la misma forma que sucede con los dominios y las direcciones IP de las páginas web.

ENS se utiliza para registrar nombres y resolver una amplia variedad de recursos, por ejemplo, que distintos tipos de registros, como pueden ser las direcciones de criptomonedas, cuyo formato está orientado a la interpretación por parte de las máquinas, se resuelvan en direcciones fáciles de recordar para los humanos. El estándar inicial de ENS define la resolución solamente para direcciones de Ethereum. Sin embargo, este estándar se diseñó para ser extendido fácilmente, con lo que en la actualidad ya permite la resolución de otros tipos de recursos, y la resolución de muchos otros podrá incorporarse en el futuro sin la necesidad de actualizar componentes centrales de ENS.

El objetivo de ENS es resolver identificadores legibles por las máquinas a identificadores legibles por los humanos, aunque también es posible realizar la conversión al revés si así lo habilita el usuario. Actualmente, se pueden usar nombres registrados en el ENS para resolver:

- Direcciones de criptomonedas.
- *Hash* de IPFS (sistema de archivos interplanetario, o *InterPlanetary File System*, en inglés, es un protocolo y red P2P destinado a almacenar y compartir datos en un sistema de archivos distribuido, similar al protocolo Swarm explicado anteriormente).
- *Hash* de Swarm.

Inicialmente, el ENS solo es compatible con los dominios terminados en .eth, .xyz, .luxe y .kred, los cuales han sido aprobados de forma manual por parte de la gestión raíz del ENS. La gestión raíz de ENS es inicialmente propiedad de cuatro de siete multifirmas con miembros de proyectos relacionados con ENS como titulares principales.

Técnicamente es posible extender este listado de dominios, pero para evitar la aprobación manual por parte de la gestión raíz de ENS, esto requiere el desarrollo de algún proceso de toma de decisiones distribuido, aún por decidir, que no pase por utilizar la multifirma que se usa actualmente. En paralelo, y con el fin de seguir una filosofía más descentralizada, en estos momentos se está planificando el despliegue de un contrato inteligente genérico, es decir, que permita a cualquiera configurar un dominio entre los más de mil domi-

nios posibles sin aprobación manual de la gestión raíz de ENS. Esta cantidad de dominios viene determinada por la cantidad de dominios que operan con extensiones de seguridad para el sistema de nombres de dominio (DNSSEC).

3.5.3. Oráculos

Una de las limitaciones principales de la arquitectura actual de los sistemas *blockchain* es que los contratos inteligentes son incapaces de consultar o iniciar una interacción con el mundo exterior fuera de la *blockchain*. Esto representa un hándicap, ya que la mayoría de sistemas necesitan información externa.

Para resolver esto se utilizan los oráculos, que son servicios de terceros que proporcionan la información externa necesaria al contrato inteligente. Los oráculos pueden verse como el enlace de conexión entre dentro y fuera de la cadena.

Por ejemplo, podríamos imaginar una aplicación de seguros descentralizada desplegada en la *blockchain* de Ethereum mediante distintos contratos inteligentes. Esta aplicación necesita de una fuente externa de datos para conocer distintos sucesos que puedan afectar a los asegurados, tales como inundaciones, sequías o diferentes catástrofes. Un agricultor está asegurado por la aplicación, y un oráculo notifica los días de lluvia que habrá la semana que viene, esta información se incorpora a los contratos inteligentes que, mediante las funcionalidades diseñadas en ellos, cubrirán al agricultor en caso de sequía.

Un segundo ejemplo puede ser aquel donde se tiene una casa de apuestas descentralizada desplegada en la *blockchain* de Ethereum mediante distintos contratos inteligentes. Los usuarios hacen sus apuestas basadas en un evento deportivo, como puede ser un partido de fútbol. Unos apuestan por el equipo A y otros por el equipo B. Un oráculo consulta el resultado del partido y lo incorpora al contrato inteligente para que reparta el premio entre los usuarios que hayan acertado el resultado.

Chainlink y Provable son dos servicios que permiten la implementación de oráculos en contratos inteligentes desplegados en Ethereum.

En Antonopoulos (2018b) se puede ver un listado más completo acerca de ejemplos y casos de uso sobre oráculos.

3.5.4. Soluciones de segunda capa

La capa 2 hace referencia a cualquier solución o mecanismo que no esté basado en el protocolo o cadena principal de Ethereum, conocido como capa 1, con el fin de mejorar la escalabilidad de las aplicaciones gracias a que las transacciones se realizan fuera de la cadena principal de Ethereum.

Esto surge debido a que, en ocasiones, el estado de la red puede perjudicar la ejecución normal de una aplicación y producir retardos y una experiencia de usuario que puede perjudicar el rendimiento de la aplicación. A su vez, la alta demanda en la red producirá un aumento en las tasas de las transacciones, lo que hará que el uso de una aplicación sea más costoso de lo normal.

Algunas de estas soluciones son los *state channels*, cadenas laterales (*sidechains*) o plasma, entre otras.

- ***State channel***: permite a los usuarios realizar transacciones entre ellos fuera de la cadena disminuyendo el uso de las operaciones en cadena y, únicamente, enviando dos transacciones a la cadena principal, la apertura y el cierre del canal. Un ejemplo es la Raiden Network (red muy parecida a la Lightning Network para Bitcoin).

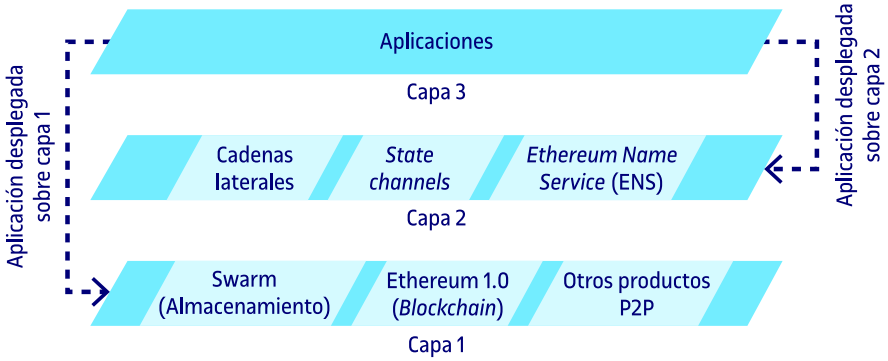
- **Cadena lateral**: cadenas de bloques independientes de la red principal que funcionan con su propio algoritmo de consenso y se conectan a la cadena principal mediante un puente (*bridge*) de entrada y salida. Un ejemplo es la red POA.

- **Plasma**: cadenas de bloques conectadas a la cadena principal de Ethereum enfocadas a pagos y liquidaciones. El caso de uso es más limitado que las cadenas laterales, ya que la seguridad es mayor, dado que estas cadenas publican en Ethereum la raíz de cada bloque plasma. Un ejemplo es la red OMG.

Es importante remarcar que no se deben confundir este tipo de soluciones con Ethereum 2.0, ya que es otra alternativa para resolver los problemas de escalabilidad de Ethereum, pero a partir de modificar directamente el protocolo.

Por último, como puede apreciarse en la figura 6, existen aplicaciones que funcionan directamente sobre el consenso de Ethereum, es decir, sobre la red principal, y otras que optan por desplegarse directamente sobre las soluciones de segunda capa. Además, las aplicaciones suelen hacer uso de otros protocolos, como pueden ser el almacenamiento de archivos (Swarm), sistema de nombres (ENS) u oráculos para consultar información *off-chain*.

Figura 6. Capas del ecosistema Ethereum



Fuente: elaboración propia

4. Las transacciones en Ethereum

Las transacciones son mensajes firmados originados por una cuenta, transmitidos por la red y registrados en la cadena de bloques de Ethereum. Las transacciones desencadenan un cambio de estado, o bien hacen que un contrato se ejecute en el EVM, es decir, que los contratos inteligentes no se ejecutan por sí solos. Ethereum no se ejecuta de forma autónoma. Todo comienza con una transacción.

4.1. Estructura de una transacción

Cada cliente y aplicación que reciba una transacción la almacenará en memoria usando su propia estructura de datos interna, quizás completada con metadatos que no existen en la propia transacción.

Una transacción es un mensaje binario serializado que contiene los siguientes datos:

- **Nonce**: un número de secuencia utilizado para evitar que el mensaje se repita.
- **Gasprice**: el precio del gas (en wei) que el programador está dispuesto a pagar.
- **Gas limit**: la cantidad máxima de gas que el programador está dispuesto a comprar para esta transacción.
- **Recipient**: la dirección de destino en Ethereum.
- **Value y data**: la cantidad de ETH para enviar a destino y los datos binarios de longitud variable.
- **V, r, s**: tres componentes que se corresponden con la firma digital ECDSA de la transacción y que sirven para obtener el remitente de la transacción.

4.2. Los pagos en Ethereum: ether y gas

Etherum utiliza la criptomoneda *ether* (ETH) para pagar por el uso de la plataforma. Así, cada vez que un usuario quiere ejecutar una acción, como realizar una transacción o ejecutar un contrato inteligente, entonces tiene que pagar con ETH por ello. Sin embargo, a diferencia de lo que pasa con Bitcoin, donde las comisiones de las transacciones se pagan directamente con bitcóin, en Ethereum se utiliza un *token* llamado *gas* para desacoplar el precio que pueda

tener el ETH en los mercados de intercambio y el coste real de las diferentes operaciones. Más abajo vemos más detalles sobre este aspecto. Sin embargo, para empezar, vamos a estudiar las diferentes unidades y denominaciones que puede tener el *ether*. Como puede apreciarse en la tabla 2, un ETH puede fraccionarse en unidades mucho más pequeñas, siendo el wei la menor unidad del sistema, equivaliendo un ETH a 10^{18} weis. Este sistema es similar al que tiene Bitcoin, donde un bitcóin equivale a 10^{8} satoshis, o al sistema del dólar, el cual puede dividirse en centavos de dólar.

Tabla 2. Denominaciones de *ether*

Nombre	Valor wei	Número de weis
Wei (wei)	1 wei	1
Kwei (babbage)	1e3 wei	1.000
Mwei (lovelace)	1e6 wei	1.000.000
Gwei (shannon)	1e9 wei	1.000.000.000
Twei (szabo)	1e12 wei	1.000.000.000.000
Pwei (finney)	1e15 wei	1.000.000.000.000.000
Ether (buterin)	1e18 wei	1.000.000.000.000.000

Fuente: Investopedia / Carla Tardi

Como hemos comentado anteriormente, la concepción del ETH fue para ejecutar acciones en la plataforma de Ethereum. Sin embargo, si utilizáramos *ether* directamente para pagar por cada tipo de acción que podemos realizar, entonces, la fluctuación del precio del ETH podría representar un problema importante para la viabilidad de la plataforma. Por ejemplo, imaginemos que en 2015, al inicio de Ethereum, se hubiera establecido que el coste de una sencilla operación aritmética, como la suma, tenía un coste de 0,01 ETH. En esos momentos, un ETH cotizaba por debajo del dólar, supongamos que 1 ETH se pagaba a 0,7 $ y, por lo tanto, 0,007 $ (0,01 x 0,7) podría haber sido un precio aceptable para una operación de suma. Sin embargo, en enero del 2021, cuando el precio del ETH superó los 1.400 $, estaríamos en un punto donde no se podría ejecutar ni una suma, ya que costaría unos 14 $. Principalmente, este es el motivo por el cual se introdujo el concepto de *gas*.

4.2.1. Sobre los límites del gas

El **gas se consume cuando se ejecuta un contrato inteligente**, es como el combustible en la computación en la cadena de bloques Ethereum. El gas es una unidad para medir la cantidad de esfuerzo computacional para una operación. Cuanto más compleja sea la operación, más gas consumirá. Simplemente, cuando el usuario transfiera ETH o ejecute un contrato inteligente, deberá pagar gas.

El objetivo del gas es mitigar la volatilidad de ETH, dado que, como ETH cotiza en los mercados de intercambios de criptodivisas, está directamente expuesto a la volatilidad del mercado. Por el contrario, el gas se usa tan solo dentro de la ejecución de un contrato y no puede extraerse hacia una EOA y, por lo tanto, no puede ni enviarse entre usuarios ni cotizar en ningún mercado de intercambio, por lo que se ve **menos** afectado por la volatilidad del mercado.

Como hemos visto en el apartado 4.1, en la estructura de las transacciones existe un campo denominado *Gas Limit* y otro *Gas Price*. Con estos dos campos, los usuarios definen cuánto están dispuestos a pagar por realizar una transacción en Ethereum. En el campo Gas Limit, el remitente establece el valor máximo de gas que está dispuesto a gastar y en Gas Price establece el precio que está dispuesto a pagar por cada unidad de gas. La multiplicación de estos dos valores da como resultado el valor máximo en ETH que costará la ejecución de esta transacción. Entonces, cuando el usuario envíe la transacción a Ethereum, deberá enviar también dicha cantidad de ETH para pagar por el coste de la ejecución. Antes de empezar la ejecución, los ETH enviados se convertirán en gas, y el nodo minero que ejecute las acciones vinculadas a la transacción irá consumiendo el gas establecido por el coste de cada operación. Si en algún punto de la ejecución se produce un error o se consume el gas máximo establecido, entonces se cancelará dicha ejecución y, si no se había consumido todo el gas, se devolverá al remitente el gas sobrante transformado a ETH. Si no se produce ningún error y la transacción tiene éxito, el gas no gastado también se devolverá al remitente en ETH.

El precio del gas se expresa en las unidades **Gwei (nanoETH = 10^{-9} ETH)**. Como puede verse en la tabla 2, el Gwei es la abreviatura de Gigawei, o 1.000.000.000 wei.

Al especificar un límite de gas, se deben considerar los siguientes puntos:

- Diferentes operaciones tendrán diferentes costos de gas.
- Los mineros reciben gas instrucción a instrucción, y dejarán de ejecutar el contrato inteligente en el momento en que se acabe el gas.
- Si queda gas sobrante, se reembolsará inmediatamente al remitente de la transacción.

En la tabla 3 se muestran los costes para las principales operaciones en Ethereum.

Tabla 3. Costes de las principales operaciones en Ethereum

Instrucción	Gas	Descripción
ADD/SUB	3	Operación aritmética
MUL/DIV	5	

Fuente: elaboración propia

Instrucción	Gas	Descripción
ADDMOD/MULMOD	8	
AND/OR/XOR	3	Operación lógica bit a bit
LT/GT/SLT/SGT/EQ	3	Operación de comparación
POP	2	Operación de pila
PUSH/DUP/SWAP	3	
MLOAD/MSTORE	3/3	Operación de memoria
JUMP	8	Salto no condicional
JUMPI	10	Salto condicional
SLOAD	800	Operación de almacenamiento
SSTORE	5.000/20.000	
BALANCE	700	Obtener el saldo de una cuenta
CREATE	32.000	Crea una nueva cuenta usando CREATE
CALL	25.000	Crea una nueva cuenta usando CALL

Fuente: elaboración propia

Veamos un escenario hipotético:

Supongamos que queremos sumar dos números, para ello el contrato debe realizar las siguientes acciones:

- **Almacenar el primer valor en la pila.** Para esta operación se utiliza el código de operación PUSH, que tiene un coste de 3 unidades de gas.

- **Almacenar el segundo valor en la pila.** Para esta operación se utiliza nuevamente el código de operación PUSH, que tiene un coste de 3 unidades de gas.

- **Sumar los dos valores que se encuentran en la pila.** Para esta operación se utiliza el código de operación ADD, que realiza la suma de los dos primeros elementos de la pila y aloja el resultado de la operación en la primera posición de la pila. Este código de operación tiene un coste de 3 unidades de gas.

Además, el remitente especifica un límite de gas de 50 unidades (Gas limit = 50) para realizar la operación y, además, el gas disponible en el bloque en el que se incluirá la transacción es de 100 unidades. Recordemos que el gas disponible en el bloque indica el total máximo de gas que se podrá gastar sumando lo que gasten el conjunto de transacciones que se incluyan en dicho bloque.

Por lo tanto, tenemos que el gas total utilizado para realizar la operación es de: 3 + 3 + 3 = 9 unidades de gas, que equivalen al coste de los códigos coste (PUSH) + coste (PUSH) + coste (ADD).

Dado que el remitente ha establecido un límite de 50 unidades de gas para realizar la transacción, sobrarán 41 unidades de gas, al ser la diferencia entre el límite de gas y el utilizado para ejecutar el cálculo.

Finalmente, el minero, a la hora de realizar el cálculo, tiene en cuenta el gas disponible en el bloque, en este caso 100 unidades de gas. Por lo tanto, hay gas disponible suficiente para poder realizar la operación de la suma de dos valores, por lo que quedan 100 - 9 = 91 unidades de gas restantes para las demás transacciones que se incluyan en este bloque.

En cuanto a los costes, lo que el remitente deberá pagar al minero que valida dicha computación queda definido según lo que se especificó en el campo Gas Price al enviar la transacción. Asumiendo que el remitente especificó un Gas Price donde se pagan 20 Gwei por cada unidad de gas, entonces la ejecución de esta transacción tendrá un coste total de 9 unidades de gas multiplicado por 20 Gwei, que resulta en 180 Gwei, es decir, 0,00000018 ETH.

Debido a que el remitente había establecido 50 unidades de gas como límite para realizar el cálculo, se le reembolsarán las 41 unidades de gas en forma de ETH, ya que no ha sido utilizado para el cálculo. Por tanto, serán 0,00000082 ETH (41 unidades de gas por 20 Gwei).

Si el remitente hubiese establecido un límite de gas de 5 unidades, el minero habría detenido la ejecución al llegar al límite finalizando la ejecución, dado que el usuario no ha establecido las suficientes unidades de gas necesarias para que se realice la ejecución de toda la operación. Por lo tanto, se revierte cualquier posible cambio de estado y se notifica al remitente mediante un mensaje de error de que no se ha podido realizar la operación.

También se debe tener en cuenta que si el gas disponible en el bloque es de 100 unidades y la operación que se va a ejecutar tiene un coste de 120 unidades de gas, aunque el usuario haya establecido 120 unidades de gas para que se pueda realizar la operación correctamente, el resultado será el mismo que en la situación anterior. La ejecución se detiene, dado que el minero detecta que no hay unidades de gas disponibles en el bloque, por lo que no puede continuar la ejecución, por lo tanto, cualquier cambio de estado se revierte y el remitente recibirá el mensaje de error de que no se ha podido realizar la operación.

Ante las dos últimas situaciones descritas en los párrafos anteriores, cuando una ejecución falla debido a que no hay suficiente gas para completar la transacción, hay que tener en cuenta que la transacción sí fue ejecutada por el minero, aunque no de forma completa, y cualquier gas utilizado antes de

que la transacción haya fallado debe pagarse. Con base en esto, la transacción revertida se incluye en el bloque que se publica en la red, pero el estado de los contratos inteligentes no se modifica.

Teniendo todo esto en cuenta, hay dos escenarios que un programador de contratos inteligentes debe considerar:

- **El límite de gas especificado es demasiado bajo**: se pierde el gas aportado y no se termina la ejecución del contrato inteligente.

- **El límite de gas especificado es demasiado alto**: aparte de acaparar recursos que no se van a usar, por la manera en que los mineros seleccionan transacciones que hay que validar, las que son más grandes tienen dificultades para encontrar mineros que les den espacio computacional. Por eso, un límite de gas demasiado alto resulta en contratos inteligentes ineficientes.

Así pues, es mejor tener una idea *a priori* de cuánto gas gastará cada contrato inteligente y poner algo de margen (de 10 a 20 %) al alza en su límite.

4.2.2. Mecanismo de valoración del precio de gas

En este punto, cabe preguntarnos cómo un remitente calcula el precio del gas en ETH que debe asignar a Gas Price al hacer una transacción. En el apartado anterior, para los ejemplos hemos asignado un valor a este concepto sin entrar en detalles, pero la verdad es que decidir un precio correcto para cada unidad de gas no es trivial, y hay que tener en cuenta que esto variará dependiendo de la oferta computacional (es decir, cuántos mineros están dispuestos a realizar cálculos) y la demanda computacional (es decir, cuántas transacciones hay en cola de ser procesadas) de la red de Ethereum. Así, cuantas más transacciones haya en espera para ser procesadas en la cadena de bloques de Ethereum, el remitente de la transacción o el propietario del contrato inteligente deberá pagar mayores tarifas de transacción. Como ya hemos comentado, el gas es el incentivo que tienen los mineros para contribuir con sus recursos computacionales a la red de Ethereum, ya que son los mineros los que se quedan los *ether* gastados en gas en la ejecución de los contratos inteligentes. Por lo tanto, en momentos de alta saturación de la red, cuando hay muchas transacciones en espera, los mineros pueden escoger procesar las transacciones que les reportan más *ether*. Así, para ganar gas de manera eficiente, los mineros prefieren e intentan extraer transacciones con tarifas altas. Por lo tanto, para ganar más gas, los mineros tenderán a hacer lo siguiente:

- Aumentar el número de transacciones que hay que validar.
- Procesar las transacciones que tienen la tarifa de transacción más cara.

Así pues, los programadores al programar contratos inteligentes (o incluso los usuarios al enviar transacciones):

- Si deciden pagar una tarifa de **transacción barata, deberán esperar bastante tiempo** hasta que un minero decida procesar su transacción.
- Si desean que **la transacción se ejecute rápidamente, deben pagar una tarifa de transacción más alta.**

Como hemos comentado anteriormente, actualmente, Ethereum tiene un problema de escalabilidad, hecho que hace que en momentos de alta demanda, el precio del gas sea muy elevado. Esto ocurre cuando en un periodo de tiempo concreto los mineros reciben muchas transacciones porque muchos usuarios están haciendo muchas transferencias de *ether* o están creando y ejecutando contratos inteligentes. Uno de los momentos de saturación de la red de Ethereum en que las comisiones fueron más altas fue a finales de 2017, que fue cuando se lanzó y popularizó el juego CryptoKitties. Este juego, que permitía a los jugadores poseer gatitos digitales y cruzarlos entre ellos, generaba un gran número de transacciones que logró congestionar la red, lo que hizo que las transacciones de todos los usuarios tardaran mucho tiempo en procesarse e incrementó el precio de las comisiones hasta las decenas de dólares por transacción.

4.2.3. Críticas al gas de Ethereum

Aunque el sistema de gas ha recibido elogios por presentar un mecanismo de funcionamiento sólido que incentiva a los mineros de manera positiva a validar las computaciones, también ha sido objeto de crítica por resultar demasiado caro para los desarrolladores y los creadores de contratos inteligentes.

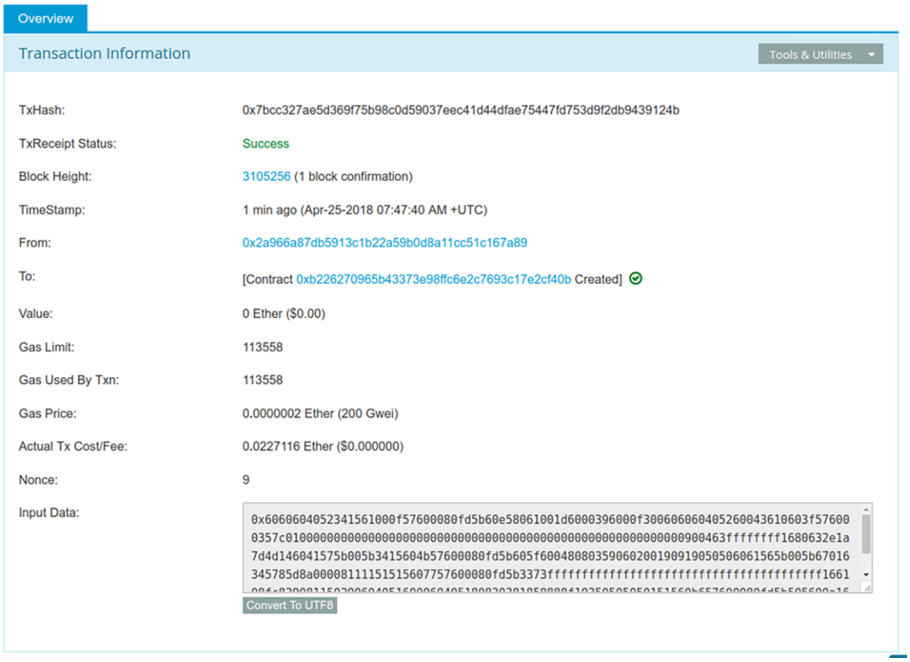
Si hacemos una comparación con un sistema AWS estándar, en AWS se pueden sumar dos números un millón de veces usando Python en 0,04 segundos, que con la tarifa aproximada al uso de 0,0059 € por hora de Amazon, EC2 cuesta 0,000000066 € en 2021. La misma computación en Ethereum puede llegar a ser 400 millones de veces más cara.

Para evitar aplicaciones descentralizadas demasiado caras, los desarrolladores deben conocer estos costes y diseñar contratos inteligentes en consecuencia. Una de las buenas prácticas para ahorrar costes es almacenar en la *blockchain* tan solo la información necesaria para aprovecharse de las capacidades de descentralización e integridad de Ethereum. El resto de información que necesite la aplicación puede almacenarse en otros sistemas fuera de la cadena (denominado popularmente en inglés como *off-chain*) (Rosic, 2021).

4.3. Consulta de la ejecución de una transacción

Ethereum es una *blockchain* pública y que puede consultarse sin requerir permisos especiales. Por lo tanto, existen herramientas en línea para inspeccionar la *blockchain* y visualizar el resultado de la ejecución de cualquier transacción hecha con un contrato inteligente. Una de las herramientas más populares para ello es Etherscan. La figura 7 muestra una captura de pantalla de esta herramienta con información sobre una transacción. Entre ellos, podemos ver el gas consumido, el Gas Limit, el Gas Price, el *nonce*, la dirección del destinatario, la dirección del remitente, etc.

Figura 7. Elementos de una transacción mostrados en el explorador Etherscan



Fuente: Etherscan

5. Creación, publicación y ejecución de contrato inteligente

Como hemos comentado anteriormente e ilustrado en la figura 4, son necesarios tres pasos básicos para poder publicar un contrato inteligente en Ethereum. Primero, para su **creación** necesitamos usar un editor y codificar el contrato en un lenguaje de alto nivel, como Solidity. Después, necesitamos **compilar** el contrato y generar el *bytecode*. Finalmente, con el *bytecode* ya podemos enviar una transacción para su **publicación** en la *blockchain*.

De hecho, la **publicación** de un contrato inteligente en Ethereum es un caso especial de transacción. Las transacciones de creación de contrato se envían a una dirección de destino especial llamada *dirección cero* (0x0). Esta dirección no representa una EOA, sino que es una dirección que nunca puede gastar el *ether* recibido ni iniciar ninguna transacción. Esta dirección solo se utiliza como destino, con el significado especial de «crear este contrato».

Una transacción de creación de contrato solo necesita contener una carga útil de datos que contenga el código compilado que creará el contrato. El único efecto de esta transacción es la creación del contrato. Además, en la transacción es posible incluir opcionalmente una cantidad de ETH en el campo de valor si se desea establecer el nuevo contrato con un saldo inicial. Si se envía un valor (ETH) a la dirección de creación del contrato sin una carga útil de datos (sin contrato), entonces es como que no hay contrato para acreditar, el ETH se pierde.

Cada contrato inteligente se **ejecuta** por los mineros de la *blockchain* de forma independiente para registrar el último estado del programa, como resultado del procesamiento de transacciones enviadas por el usuario. A diferencia de las bases de datos tradicionales, las cadenas de bloques están descentralizadas. Cada nodo asume que otros son potencialmente maliciosos y nunca confía en los estados mantenidos por otros nodos en la red. En cambio, cada nodo mantiene su propia base de datos de estado ejecutando las transacciones ellos mismos utilizando el código del contrato inteligente, y el estado final se decide por consenso.

Para ejecutar el contrato inteligente, se crea una transacción con una función, pasando por parámetro los valores de entrada para la llamada a la función y siempre firmada por el remitente. Se puede enviar una transacción a cualquier nodo de la red *blockchain*, que la transmite a toda la red para que todos los nodos vean la transacción. En algún momento, la transacción es procesada por cada nodo individual utilizando el programa ejecutable en el contrato inteligente de destino. Si se ejecuta con éxito la transacción, se actualizará

el estado interno de la cadena de bloques. Un contrato inteligente también puede considerar que la entrada no es válida y rechazar la transacción como fallida, en cuyo caso, su estado no se ve afectado.

Veamos (figura 8) un ejemplo sencillo, en Solidity, de un contrato inteligente con dos definiciones: una primera para **emitir** (*mint,* en inglés) moneda virtual, y una segunda para **hacer transferencias** entre dos direcciones, que significa que se resta un importe en la cuenta de emisión y se suma dicho importe en la cuenta destino.

Figura 8. Ejemplo de contrato inteligente con dos definiciones

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

contract Coin {
  // El keyword «public» hace que las variables minter y balances
  // sean legibles fuera del smartcontract
  address public minter;
  mapping (address  => uint) public balances;

  // Los eventos permiten a los clientes reaccionar eficientemente
ante cambios en el contrato
    event Sent (address from, address to, uint amount);

  // Este es el constructor que se ejecuta solo cuando se crea el
smart contract
  // Anotamos como propietario quien despierta este smartcontract
  constructor() {
      minter = msg.sender;
  }

  // Con mint creamos la cantidad de moneda virtual (amount) que
queramos
  // y controlamos que el minter sea el propietario de este smart
contract
  // (si lo llama otra dirección distinta de la del propietario, pues
simplemente ignora la petición).
  function mint (address receiver, uint amount) public {
      require (msg.sender == minter);
      require (amount < 1e60);
      balances[receiver] += amount;
  }

  // Y ahora ya podemos enviar moneda en la cantidad (amount) que
queramos
  // desde la dirección de quien invoca a esta función «send» a la
dirección de destino
  // y controlando que tengamos saldo suficiente.
  function send (address receiver, uint amount) public {
      require (amount <= balances[msg.sender], "Saldo insuficiente.");
      balances[msg.sender] -= amount;
      balances[receiver] += amount;
      emit Sent (msg.sender, receiver, amount);
  }
}
```

Fuente: elaboración propia

6. Las DApps (aplicaciones descentralizadas)

Las aplicaciones descentralizadas (DApps) son programas de software cuyo funcionamiento se mantiene mediante una red distribuida de nodos informáticos en lugar de un solo servidor y, por eso, Ethereum es una buena opción para implementar la base de una DApp.

En general, las aplicaciones tienen dos partes: una parte frontal, que se denomina *frontend* (o 'cliente'), como por ejemplo una interfaz web, y por otra parte el *backend* (o 'servidor') con la capa de acceso a los datos y la ejecución de la funcionalidad de la aplicación.

La principal diferencia con las aplicaciones descentralizadas se encuentra en que, en las aplicaciones convencionales, tanto el *backend* como el *frontend* están alojadas en un solo servidor. Por el contrario, el *backend* de las DApps se aloja en una red distribuida de servidores sincronizados repartidos por todo el mundo. Por su parte, el *frontend* puede distribuirse como un fichero ejecutable que cada usuario puede instalar en su dispositivo o puede habilitarse una interfaz web mediante protocolos de almacenamiento distribuidos, como Swarm.

La primera aplicación descentralizada fue un software de intercambio de archivos P2P llamado Napster, creado en 1999, diez años antes de que apareciera la primera *blockchain*. Otros ejemplos de aplicaciones descentralizadas que se ejecutan en redes P2P, pero que no usan una *blockchain*, son el popular servicio de intercambio de archivos BitTorrent y el navegador Tor, de código abierto, centrado en la privacidad.

Sin embargo, gracias a las plataformas como Ethereum, las aplicaciones descentralizadas han experimentado un gran auge, y es cuando se ha popularizado el término de DApp. Las DApps tienen su base en la creación de uno o varios contratos inteligentes, que recordemos que una vez desplegados en la *blockchain*, se ejecutan automáticamente mediante transacciones y de acuerdo con sus términos predefinidos en el momento de su publicación. De esta forma, el contrato inteligente pasa a ser una pieza que puede ser monitoreada y verificada por cualquiera y no requiere que los usuarios confíen en terceros para su correcto funcionamiento. En ese sentido, las DApps se implementan como un grupo de contratos inteligentes interoperables que funcionan con una interfaz de usuario común. En términos de funcionalidad, las DApps deberían alcanzar el mismo servicio y la misma experiencia de usuario que las aplicaciones convencionales, pero siendo ejecutadas por medios descentralizados y, por lo tanto, aportando otras características como:

- **Resistencia a la censura**: debido a que se ejecutan sobre redes abiertas y sin permisos, ninguna entidad tiene el poder o la autoridad para impedir que los usuarios accedan o implementen aplicaciones descentralizadas.

- **Transparencia**: dado que las aplicaciones descentralizadas se ejecutan en *blockchains* públicas y transparentes, todos los datos, incluido el código fuente de la DApp y todas las transacciones entrantes y salientes, también son transparentes y están disponibles públicamente. Esto añade seguridad, porque todo el comportamiento en cadena (*on-chain*) es perfectamente verificable, y el código de las DApps puede ser revisado y auditado por cualquier persona en cualquier momento.

- **Eliminación del tiempo de inactividad**: uno de los mayores beneficios de ejecutar aplicaciones en redes informáticas distribuidas es que cuando un nodo cae o falla un componente, todas las unidades restantes siguen funcionando y pueden asumir el trabajo, con lo que se eliminan puntos únicos de fallo. De esta manera, una vez que el contrato inteligente en el corazón de la aplicación se despliega en la *blockchain*, la aplicación continuará ejecutándose ininterrumpidamente mientras la red esté activa. Además, debido a su naturaleza descentralizada, las DApps son resistentes a muchas amenazas de seguridad, incluidos los ataques de denegación de servicio distribuidos (DDoS), las inyecciones SQL (una vulnerabilidad de seguridad web que permite a un atacante interferir con las consultas), las bombas XML (un mensaje redactado y enviado con la intención de sobrecargar un analizador XML) y las secuencias de comandos entre sitios web (agujeros de seguridad basados en la explotación de vulnerabilidades del sistema de validación), que son de lo que padecen las aplicaciones web habituales centralizadas.

- **Privacidad**: en general, los usuarios pueden interactuar libremente con las DApps utilizando una *wallet* de criptomonedas y no necesitan proporcionar ni revelar ninguna información de identificación personal.

- **Innovación abierta**: en general, las DApps se implementan siguiendo una filosofía de código abierto que facilita la innovación.

Asimismo, las DApps también tienen varios inconvenientes, comparadas con las aplicaciones convencionales:

- **Propensas a errores y hackeos**: el riesgo de ejecución de contratos inteligentes es definitivamente el primero de la lista de inconvenientes. Si bien la ejecución determinista y autónoma del código y la inmutabilidad de la *blockchain* tiene ventajas de seguridad, puede causar estragos si se ejecuta mal. Incluso los errores de codificación más pequeños pueden resultar en DApps que funcionen gravemente mal, y los errores de diseño más imper-

ceptibles pueden conducir a usos dañinos, como la pérdida de fondos bloqueados en los contratos o la congestión hasta el punto de la inutilización.

- **Difíciles de construir**: la inmutabilidad de los contratos inteligentes hace que construir y diseñar DApps sea especialmente difícil. Los programadores las han de planificar exhaustivamente y prepararlas a prueba de futuro, porque una vez que se ejecutan sus contratos inteligentes, hacer cambios en ellos es, relativamente, imposible.

- **Experiencia de usuario inferior**: por ahora, las DApps proporcionan una experiencia de usuario inferior que las aplicaciones centralizadas. Por ejemplo, la navegación en una DApp suele ser menos intuitiva, incluso para usuarios avanzados. Además, si un usuario realiza una transacción incorrecta en una DApp, se ofrecen pocas soluciones para revertirla, dado que las transacciones de *blockchain* son irreversibles y definitivas, lo que no deja lugar a errores y los errores pueden costarles caro a los usuarios.

- **Pueden ser caras**: como hemos introducido en secciones anteriores, realizar una transacción en una aplicación descentralizada, si está ejecutada en Ethereum, requiere pagar una tarifa de transacción de red. Esta tarifa de transacción, calculada en gas y liquidada en ETH, va directamente a los mineros que mantienen la red *blockchain* en la que se ejecuta la DApp. En tiempos de congestión de la red Ethereum, las transacciones simples pueden costar varios euros, y las transacciones más elaboradas pueden llegar a resultar muy caras respecto a las aplicaciones centralizadas.

- **Lentas**: además de ser caras, las DApps también son más lentas que las aplicaciones convencionales. Esto se debe a que la *blockchain* de Ethereum permite por ahora unas 15 transacciones por segundo. Incluso Ethereum 2.0, donde podrán ejecutarse varios miles de transacciones por segundo, seguirá por debajo del rendimiento de las aplicaciones centralizadas. Esto significa que cuando el uso excede la capacidad de la red, la *blockchain* se congestiona y las transacciones cuestan más y tardan más en liquidarse. En ese caso, a menos que los usuarios paguen una tarifa de gas extremadamente alta, sus transacciones pueden tardar horas en liquidarse o incluso pueden terminar rechazadas por la red, lo cual genera imprevisibilidad en la ejecución.

6.1. Ejemplos de DApps

Dos de los tipos de DApps más populares en estos momentos son: los intercambios descentralizados (DEX) y las organizaciones autónomas descentralizadas (DAO).

6.1.1. Intercambios descentralizados (DEX)

Los intercambios descentralizados y los protocolos de intercambio de *tokens* son las DApps más utilizadas en el espacio de las criptomonedas. Los intercambios descentralizados utilizan contratos inteligentes o las propias *wallets* de los usuarios en vez de intermediarios para custodiar los fondos, lo que reduce el riesgo de piratería en el intercambio y el robo de fondos de custodia. Todas las transacciones en intercambios descentralizados son *peer-to-peer* o *peer-to-contract*, y los fondos van directamente de las *wallets* de los usuarios a los contratos, y a la inversa, sin pasar por las manos de terceros.

En los primeros intercambios descentralizados se llevaban a cabo contratos inteligentes que replicaban la lógica de los libros de órdenes de los intercambios centralizados donde los *traders* añaden sus órdenes de venta y compra de valores. Posteriormente, los DEX han evolucionado y utilizan otras fórmulas para poner de acuerdo a compradores y vendedores. Una de estas fórmulas son los creadores de mercado automatizados (*automatic market makers*, o AMM, en inglés), que son protocolos que utilizan contratos inteligentes para crear fondos de liquidez entre pares de *tokens* siguiendo algoritmos preestablecidos o fórmulas matemáticas para determinar el precio de un *token* en función del otro.

Algunos de los mercados descentralizados más populares son Uniswap y Bancor.

6.1.2. Organizaciones autónomas descentralizadas (DAO)

Las organizaciones autónomas descentralizadas, o DAO (*decentralized autonomous organisations*, en inglés), son precisamente lo que describe su nombre. En lugar de gestionarse con las jerarquías típicas de las organizaciones tradicionales, las DAO aprovechan los contratos inteligentes para ejecutar decisiones de forma autónoma.

Si bien las DAO pueden tener muchas funciones, una de las más relevantes es que permiten a los usuarios de una DApp gobernarla de manera descentralizada. Las DApps deben continuamente innovar y evolucionar para mantenerse al día y seguir siendo relevantes, y para ello se deben tomar decisiones que no pueden provenir de una sola persona o grupo, ya que esto iría en contra de los valores de la descentralización.

Para resolver este problema, una DApp puede conformar una DAO que, entre otras cosas, permite a los usuarios votar y proponer cambios de protocolo, crear tesorerías sin custodia para financiar el desarrollo futuro y otorgar a los usuarios ciertos derechos o distribuir acciones de propiedad a la DApp. En general, para poder participar de las decisiones de la DAO, los usuarios han de disponer de algún *token* especial.

Una de las DAO más populares es MakerDAO para el control de la criptomoneda Dai.

Resumen

En este módulo hemos empezado revisando el concepto de Web 3.0, que pretende apoyarse en la tecnología *blockchain* para ofrecer una web más descentralizada donde los usuarios sean menos dependientes de grandes intermediarios y puedan controlar ellos mismos sus datos.

Hemos visto que Ethereum puede ser una de las tecnologías más importantes para habilitar esta Web 3.0, ya que, a diferencia de Bitcoin, Ethereum ofrece un lenguaje de programación Turing completo con el que se pueden crear todo tipo aplicaciones. Así, los programadores pueden desplegar aplicaciones en Ethereum, que actuará como una especie de ordenador universal guardando el estado de multitud de aplicaciones diferentes en una misma *blockchain*. Para ello, Ethereum crea una red P2P donde los nodos contribuyen a distribuir las transacciones de los usuarios y los bloques de la cadena, a ejecutar los contratos inteligentes, mediante la EVM, y a guardar los resultados de las ejecuciones en la *blockchain*. En este módulo hemos descrito estos componentes y, además, también hemos visto cómo funciona el sistema monetario con el *ether* y el gas para poder desplegar y ejecutar contratos en Ethereum.

Finalmente, también hemos destacado que Ethereum es tan solo la capa básica de este ecosistema, y que a su alrededor están apareciendo otras muchas tecnologías para dar respuesta a otras necesidades de las DApps, como, por ejemplo, oráculos para consultar datos provenientes de fuera de la *blockchain* o soluciones de segunda capa para facilitar el intercambio de *tokens* digitales de forma rápida y con bajas comisiones de transferencia.

Glosario

decentralized application *f* Programas de software cuyo funcionamiento se mantiene mediante una red distribuida de nodos informáticos en lugar de un solo servidor en un protocolo descentralizado como *blockchain*.
sigla **DApp**

decentralized autonomous organization *f* Organización representada por reglas codificadas en contrato inteligente que es transparente, controlada por los miembros de la organización, gestionada y gobernada de forma descentralizada.
sigla **DAO**

decentralized exchanges *m* Mercados de intercambio de criptomonedas que permiten que las transacciones directas de criptomonedas se realicen en línea de forma segura y sin la necesidad de un intermediario.
sigla **DEX**

distributed ledger technology *f* Tecnología que habilita almacenar, compartir y sincronizar datos de manera distribuida entre todos los nodos de la red sin depender de autoridades o nodos centrales a partir de un mecanismo de consenso.
sigla **DLT**

ether *f* *Token* nativo de Ethereum.
sigla **ETH**

ethereum virtual machine *f* Entorno de ejecución encargado de la implementación y ejecución de contratos inteligentes, así como de las actualizaciones de estado de estos contratos.
sigla **EVM**

externally owned account *f* Cuenta de Ethereum que es controlada por un usuario o persona que es poseedor de la clave privada de la cuenta.
sigla **EOA**

proof of stake *f* Algoritmo de consenso basado en el poder de cómputo para resolver un problema criptográfico.
sigla **PoW**

proof of work *f* Algoritmo de consenso basado en el poder de cómputo para resolver un problema criptográfico.
sigla **PoW**

Bibliografía

Antonopoulos, A. (2018a). «What Is Ethereum?» y «Ethereum Basics». En: A. Antotonopoulos; G. Wood. *Mastering Ethereum: Building Smart Contracts and DApps* (págs. 1-40) [en línea]. Sebastopol: O'Reilly. [Fecha de consulta: 8 de enero de 2021]. <https://github.com/ethereumbook/ethereumbook/blob/develop/book.asciidoc>

Antonopoulos, A. (2018b). «Oracles: Oracle Use Cases and Examples». En: A. Antotonopoulos; G. Wood. *Mastering Ethereum: Building Smart Contracts and DApps* (págs. 254-255) [en línea]. Sebastopol: O'Reilly. [Fecha de consulta: 8 de enero de 2021]. <https://github.com/ethereumbook/ethereumbook/blob/develop/book.asciidoc>

Buterin, V. (2013). *Ethereum whitepaper* [en línea]. Ethereum. <https://ethereum.org/en/whitepaper/>

ConsenSys (2020). «Ethereum 2.0 FAQ» [en línea]. ConsenSys. [Fecha de consulta: enero de 2021]. <https://consensys.net/knowledge-base/ethereum-2/faq>

EthHub (2020). «Ethereum 2.0 Phases» [en línea]. *EthHub*. [Fecha de consulta: enero de 2021]. <https://docs.ethhub.io/ethereum-roadmap/ethereum-2.0/eth-2.0-phases/>

Miller, A.; Bentov, I. (2016). «Zero-Collateral Lotteries in Bitcoin and Ethereum» [en línea]. <https://arxiv.org/pdf/1612.05390v1.pdf>

Muzzy, E. (2020). «What Is Ethereum 2.0?» [en línea]. *ConsenSys*. [Fecha de consulta: enero de 2021]. <https://consensys.net/blog/blockchain-explained/what-is-ethereum-2/>

Nakamoto, S. (2008). «Bitcoin: A Peer-to-Peer Electronic Cash System» [en línea]. [Fecha de consulta: 1 de febrero de 2021]. <https://www.metzdowd.com/pipermail/cryptography/2008-October/014810.html>

Rosic, A. (2020). «What is Ethereum Gas? (The Most Comprehensive Step-By-Step Guide Ever!)» [en línea]. *Blockgeeks*. [Fecha de consulta: 11 de enero de 2021]. <https://blockgeeks.com/guides/ethereum-gas/>

Szabo, N. (1996). «Smart Contracts. Phonetic sciences» [en línea]. [Fecha de consulta: 14 de noviembre de 2020]. <https://www.fon.hum.uva.nl/rob/Courses/InformationInSpeech/CDROM/Literature/LOTwinterschool2006/szabo.best.vwh.net/smart_contracts_2.html>

Wood G. (2014a). «ĐApps: What Web 3.0 Looks Like» [en línea]. *Gavin Wood*. [Fecha de consulta: enero de 2021]. <https://gavwood.com/dappsweb3.html>

Wood G. (2014b). «Less-techy: What is Web 3.0?» [en línea]. *Gavin Wood*. [Fecha de consulta: enero de 2021]. <https://gavwood.com/web3lt.html>

Páginas web

OMG Network [Fecha de consulta: abril de 2021]. <https://omg.network/>

POA Network [Fecha de consulta: abril de 2021]. <https://www.poa.network/>

Raiden Network [Fecha de consulta: abril de 2021]. <https://raiden.network/>